Dynamics of Mirroring: Temporal Sequencing of Mirror-Misting and Mirror-Focusing in Open-Source Evolution

Abstract: The mirroring hypothesis posits a correspondence between ties in a team and technical dependencies in its work. Mirroring is a cornerstone in understanding the development of complex technical systems. However, its application to Open-Source Software (OSS) suggests weak support. Why does mirroring, prevalent in many complex systems, appear absent in OSS, and what alternative mechanisms might explain evolutionary outcomes in OSS and analogous distributed innovation environments? Departing from prior studies examining mirroring as a static state of the system, we focus on its unbroached *dynamics*, specifically mirror focusing and mirror misting, and their evolutionary consequences. We theorize that it is the *sequence* of these dynamics, rather than either state in isolation, which drives evolutionary outcomes. We analyze the evolving source code and commit histories of 1,279 projects over 20 years, using methods analogous to functional subsequences in genome analysis. We find that mirroring dynamics fluctuate in OSS projects, creating temporal sequences of mirror misting followed by mirror focusing. Using panel vector autoregression and fixed-effects models to analyze projects' codebases and commit-logs, we show that mirror focusing enhances project evolution across both systemwide and module-specific levels. Our findings move the mirroring discourse beyond static notions to its processual dynamics, contributing novel insights into OSS development by uncovering its antifragility, in which sequencing mirroring dynamics enhances evolutionary outcomes. To the mirroring hypothesis discourse, we add how the oscillatory sequences of misting followed by focusing--not mere achievement of static mirroring--gives rise to evolutionary benefits.

Keywords: open-source software (OSS), mirroring hypothesis, team structure, project structure, dynamics, IS evolution.

Dynamics of Mirroring: Temporal Sequencing of Mirror-Misting and Mirror-Focusing in Open-Source Evolution

1 INTRODUCTION

Mirroring refers to the correspondence between a product architecture and its developing organizational structure. In projects developed by teams such as OSS projects, this correspondence is between project architecture and team structure. The mirroring hypothesis states that this correspondence is a key driver of project performance (Colfer & Baldwin, 2016; Sanchez & Mahoney, 1996). In OSS development, the project architecture is defined by dependencies among its modules, while the team structure results from developers' collaboration, which requires coordination during development. Both can lie anywhere on a continuum ranging from perfectly monolithic to perfectly modular. A modular team working on a modular project would be mirrored; a monolithic team would not be mirrored.

The mirroring hypothesis draws on the literature on organization design and organizations as complex systems (Lawrence & Lorsch, 1967; Thompson, 1967), on product design and products as complex systems (Conway, 1968), and on the concept of modularity (Parnas, 1972; Simon, 2002). It suggests that organizational ties within a team, firm, or group of firms will correspond to the technical dependencies in the developed product. Such correspondence conserves scarce cognitive resources during problem-solving. Prior work suggests that mirrored systems tend to be more successful overall (Colfer & Baldwin, 2016). However, numerous instances exist where this is not the case (Baldwin, 2023; Colfer & Baldwin, 2016).

One notable exception to mirroring is open collaboration projects, such as OSS, for which 56% of studies do not support it (Colfer & Baldwin, 2016, p. 726). One explanation is that built-in coordination mechanisms in OSS development eliminate the necessity for collaboration (Colfer & Baldwin, 2016). A similar explanation is that new software development technologies

reduce the need for direct coordination. However, a deeper explanation could be that deviating from mirroring may be advantageous. Indeed, studies of mirroring outside the context of OSS uncovered situations where organizations could benefit by partially or wholly deviating from mirroring, a concept referred to as 'breaking' the mirror when this deviation is large and 'misting' the mirror when the deviation is small (Alochet, MacDuffie, & Midler, 2023; Baldwin, 2023; Furlan, Cabigiosu, & Camuffo, 2014). In particular, organizations benefit from misting the mirror when they want to pursue architectural innovations, when strict mirroring is prohibitively costly or logistically infeasible, or when they want to trade off the economy of strict mirroring in favor of other benefits or in response to organizational limitations and constraints (Baldwin, 2023; Burton & Galvin, 2022). Therefore, similar benefits may also be accrued in OSS development.

All information systems, including OSS, need to evolve to meet changing requirements. We focus on project evolution as an outcome that has yet to garner enough attention (Agarwal & Tiwana, 2015). The system architecture partially determines this evolution, and thus, evolution has strong path dependency (Lehman, 1979; Tiwana, Konsynski, & Bush, 2010). However, evolution is also determined by decisions made by the development teams. The mirroring hypothesis provides an integrated approach to examining evolution at the intersection between the project and the team.

Our broad research question is, "How does mirroring affect OSS evolution?" We combine these two research streams—OSS evolution and the mirroring hypothesis—and focus on unpacking the theoretical puzzle of mirroring in OSS development. Namely, if mirroring is advantageous, why do OSS projects not exhibit mirroring similar to that in other contexts? Notwithstanding the prior explanation of implicit coordination, we build on the second view,

which suggests that deviating from mirroring offers advantages and focuses on the mechanism by which this deviation is advantageous. Noting that mirroring in OSS projects is prone to fluctuations due to their open, fluid membership, we propose an alternative approach to understanding mirroring's role in OSS development. Specifically, we focus on its dynamics.

We refer to the classical notion of mirroring —the correspondence between project architecture and team structure —as *the state view of mirroring*, or simply mirroring. We refer to changes of mirroring as *the dynamic view of mirroring*, or mirroring dynamics. Mirror focusing and mirror misting are the two mechanisms undergirding mirroring dynamics, temporally increasing and decreasing the state of mirroring, respectively. The dynamics view emphasizes the importance of change as much as the state itself. Just as biological fitness depends on adaptive capacity rather than static optimality, software project success increasingly depends on the rate at which systems incorporate new functionality and adapt to changing environments more than on achieving static perfection at any given moment (Agarwal & Tiwana, 2015; Andriani & Cattani, 2024; Tiwana, 2025).

Specifically, weak mirroring in OSS may be due to its open membership and self-selection, which facilitates deviating from architectural patterns and misting the mirror as a result. However, projects that reestablish mirroring through focusing, notwithstanding misting, gain *evolutionary* advantages. We focus on evolution as the outcome of long-term development activity, which is especially important as information systems become longer-lived (Kemerer & Slaughter, 1999) and where, in a competitive landscape, systems must evolve progressively faster just to keep pace with rivals (Agarwal & Tiwana, 2015).

The departure from a state view into a dynamics view builds on Simon's (2002) systems theory idea that complex systems are shaped by both static configurations and dynamic

transitions. Mirror *focusing* functions as a homeostasis that reduces misalignment to ease coordination. Mirror *misting* introduces productive disequilibrium that enables exploration to identify superior designs (Baldwin, 2023). From modularity theory, focusing increases the correspondence between technical and organizational modularity, whereas misting disrupts this correspondence. Evolutionary benefits arise from *alternation* between states, creating antifragility, the property of systems that improve rather than merely withstand perturbations (Taleb, 2012), through controlled oscillations (Nickerson & Zenger, 2002).

We develop and empirically test three propositions and further explore how mirroring dynamics lead to project evolution. Our research context encompasses the evolving source code of 1,279 open-source projects spanning over 20 years and 280,385 person-years of development work. In a novel departure from the prevailing perceptual and static measures used in studies of mirroring and information systems (IS) evolution, we assess team and project structures by directly analyzing projects' codebases and commit logs. Our panel vector autoregression and fixed-effects econometric models account for the endogenous reciprocal impacts between team and project structures. The findings support the propositions and provide further insights into the dynamics of mirroring in OSS.

The study provides insights into OSS evolution and the mirroring hypothesis literature streams and sheds light on the complex, multidimensional processes that shape the evolution of these systems. In relation to OSS, the study highlights the complex nature of development. It surfaces the importance of unpacking dynamics to explain poorly understood evolutionary consequences in IS development (Eaton, Elaluf-Calderwood, Sørensen, & Yoo, 2015; Fürstenau, Baiyere, & Kliewer, 2019; Rolland, Mathiassen, & Rai, 2018; Tiwana, 2015). Regarding the mirroring hypothesis, the study emphasizes the importance of examining the dynamics of

mirroring (Fang & Kim, 2023). By theorizing the first and second-order effects of mirroring (Sanchez, Galvin, & Bach, 2023), the study contributes to understanding how mirroring affects system evolution.

2 THEORETICAL DEVELOPMENT

Our first theoretical foundation is modularity—the degree to which parts of any system can be separated and recombined (Schilling, 2000; Simon, 2002). Modularity is a concept that applies to various types of systems, including technological, social, and biological. In technical systems, modular design partitions functionality into distinct modules with well-defined interfaces, minimizing intercomponent dependencies while maintaining integration among them. This enables parallel development, localized changes, and evolutionary flexibility (MacCormack, Rusnak, & Baldwin, 2006). Organizational modularity decomposes work into semi-autonomous units such as sub-teams with specified interaction protocols (Hoetker, 2006). The mirroring hypothesis is about the correspondence between these two forms. Grounding our theorizing in modularity theory enables a precise conceptualization of both system architecture and team structure.

We focus on project *evolution* for four reasons. First, evolution represents the cumulative consequence of development, connecting mirroring dynamics across multiple development cycles. Unlike point-in-time performance measures, whether perceptual surveys of alignment quality or cross-sectional productivity metrics , evolution reflects the cumulative trajectory of development decisions, making it uniquely sensitive to the sequential patterns of misting and focusing that our theory foregrounds. Second, as systems become longer-lived and embedded across industries, the capacity to adapt to changing needs and new technologies drives competitive success (Agarwal & Tiwana, 2015). In OSS specifically, where projects lack centralized governance and rely on emergent coordination, evolution serves as an observable

manifestation of how well the sociotechnical system sustains its capacity for change (Lehman, 1979), one that is directly measurable from commit histories without relying on self-reports or perceptual surveys. Third, evolution is path-dependent and sociotechnically driven (Tiwana & Safadi, 2024), making it an ideal context for examining the interplay that mirroring seeks to explain. Prior architectural decisions constrain and enable future ones, thus the effects of mirroring dynamics compound rather than reset between cycles. Because mirroring operates at the intersection of project architecture and team structure, its cumulative effects are most visibly captured in outcomes that span both dimensions. This makes evolution a natural dependent variable for a theoretical model premised on sequentially-accumulating dynamics. Fourth, evolution is multidimensional. Following Henderson and Clark (1990), we further decompose evolution into system-wide and module-level changes to provide a precise dual-dimensional lens, enabling us to detect whether mirroring dynamics differentially affect how modules are reconfigured versus how their internal content changes. This distinction subsequently reveals nuanced asymmetries in the pathways through which mirror focusing operates.

2.1 OSS Evolution

Software evolution refers to changes in the system's functionality or properties (Chapin, amd Khaled Md. Khan, Ramil, & Tan, 2003). Information systems are remarkably long-lived, and most of their costs are paid during maintenance rather than initial development. The Y2K problem highlighted the cost of evolving legacy systems, and since then, software evolution has become a mainstream focus of software engineering research (Kemerer & Slaughter, 1999; Lehman, Ramil, Wernick, Perry, & Turski, 1997). Agarwal and Tiwana (2015) discuss software evolution in IS research through the lens of competitive dynamics, where systems must evolve progressively faster to keep pace with their rivals. The premium on evolvable systems is rising

dramatically as software is embedded in non-technological products, magnifying both the scale and the distributed nature of modern information systems.

Software engineering studies of evolution focus on the technological aspect of evolution. Software (Lehman et al., 1997) and the broader evolution of technology (Arthur, 2009) can exhibit laws similar to those of other natural phenomena. However, notwithstanding that general laws of evolution can be conceived at a higher-level macro scale, at a more meso-level, evolution results from the actions of development team members. Therefore, how teams coordinate and collaborate would ultimately change the software, and vice versa (Bolici, Howison, & Crowston, 2016).

Evolution is not a unidimensional concept. There are different modalities through which systems change. Henderson and Clark (1990) popularized four innovation modalities: incremental, modular, architectural, and radical. These four modes result from the architecture of the two dimensions of change: changes within modules and changes in module connections. In line with the IS development context (e.g., Tiwana & Safadi, forthcoming), we refer to them as module-level and system-wide evolution, respectively. Focusing on two dimensions of evolution enables us to capture nuances that may be obscured when considering change on a single dimension. Furthermore, prior work shows that within module code guides development activity (Bolici et al., 2016). Developers' actions change and evolve the codebase. Likewise, the architecture of the codebase (e.g., its modules and interfaces) guides developers' actions.

In sum, it is expected that the interplay of team and project structure affects project outcomes. Indeed, this is the gist of the mirroring hypothesis.

2.2 The Mirroring Hypothesis

The mirroring hypothesis posits that the product architecture and organizational structure mirror each other: "organizational ties in a project, firm, or group of firms will (or should) correspond to

the technical dependencies in the work being performed" (Baldwin, 2024, p. 35). The hypothesis has descriptive and normative perspectives. Descriptive studies examine the relationship between technical dependencies and organizational ties. In contrast, the normative perspective assumes that a designer or team intended to create a mirrored or unmirrored system and correlate that system design choice with performance outcomes. Normative studies of firms offer a more nuanced view of mirroring as an organizational choice: mirroring generally leads to good performance outcomes, but often to poor ones when strictly and rigidly adhered to (Colfer & Baldwin, 2016).

Empirical evidence suggests that mirroring can be an effective strategy for conserving cognitive resources, but there are exceptions to the hypothesis. Partial mirroring is a strategy in which firms do not strictly mirror their partners' or suppliers' technical systems, but instead adopt a more flexible approach (Colfer & Baldwin, 2016). It is effective when technological change accelerates, and systems become more complex, making strict mirroring counterproductive (Baldwin, 2023). Also, empirical studies have shown strong support for the mirroring hypothesis in the case of modularity (e.g., when a modular organization develops a modular product), but there is a need for further research on contingencies and factors that can affect this relationship (Burton & Galvin, 2022; Chen, Chowdhury, & Donada, 2019; Hao, Feng, & Frigant, 2017; Sorkun & Furlan, 2017).

2.3 Mirroring State and Dynamics

In its descriptive and normative views, mirroring is commonly viewed as the outcome of an underlying, unobserved balance of costs and benefits (Baldwin, 2023; Furlan et al., 2014; Murmann & Schuler, 2023). While most studies view mirroring as a static state, some focus on firms' deliberate decisions to increase or decrease mirroring. When mirroring is viewed as a strategic decision, it becomes an outcome that can be reinforced or diminished over time.

Prior work on IS distinguishes between the IS phenomenon as a state of affairs and as strategic actions of firms. For example, Sabherwal et al. (2019) examine the alignment between a firm's business and information technology not only as a state but also as a dynamic capability. Their findings emphasize the dual nature of alignment and the contingent role of the environment. The state aspect of alignment is advantageous across all environments but can lead to undesired rigidity in stable ones. The capability aspect of alignment is beneficial in complex environments. Our distinction between the state of mirroring and its dynamics reflects an ontological respecification of what mirroring fundamentally is: a process unfolding across time rather than a structural property fixed at the moment we observe it. This conceptually parallels Sabherwal et al.'s (2019) alignment duality of state versus capability, which operate through different mechanisms, positioning mirroring dynamics as an emergent outcome of distributed development rather than a purposeful choice. Our subsequent development theorizes pertinent mechanisms—mirror-misting and mirror-focusing; fleshes out how temporary misalignment can accelerate architectural learning rather than just being a coordination hazard; and clarifies its emergent nature arising from decentralized developers' aggregate actions.

Mirroring refers to the correspondence between the product technical architecture and the organizational structure. Therefore, building on this prior work as well as recent work on mirroring[1] we make a similar distinction between the state of mirroring and the dynamics of mirroring as a change that aims either at increasing the correspondence between the product and organization structures (i.e., mirror *focusing*) or decreasing this correspondence (i.e., mirror *misting*) (Furlan et al., 2014; Windrum, Haynes, & Thompson, 2019). Specifically, we define

[1] *Industrial and Corporate Change*'s special issue on design rules provides a summary (Brusoni et al., 2023)

mirror focusing as an intertemporal increase in the correspondence between project architecture and team structure, and mirror misting as an intertemporal decrease in this correspondence.

2.4 Importance of Mirroring Dynamics

Under a normative view of mirroring, mirroring is desirable because it aligns technical and organizational interdependencies, reduces coordination frictions and integration risks. Thus, under this mainstream normative view, mirror focusing becomes the key mechanism through which projects can intentionally or emergently restore socio-technical correspondence as they evolve and drift (Tiwana & Safadi, forthcoming, 2023). This mainstream assumption is why the literature treats mirror focusing as inherently advantageous, advising firms to make strategic decisions to increase mirroring (Colfer & Baldwin, 2016). However, many contingencies favor going in the opposite direction or misting the mirror (Fang & Kim, 2023; Sorkun & Furlan, 2017).

First, the creation of new, technically challenging products or systems often occurs at the boundaries of firms, which can increase interdependence and require greater communication and coordination among the actors involved (Colfer & Baldwin, 2016). For example, developing a new medical device requires intensive back-and-forth between the design team, hardware suppliers, and clinical partners, even when the device is designed to be modular.

Second, organizations can benefit from misting the mirror when strategically designing modules that extend knowledge beyond task boundaries, allowing for broader knowledge sharing and information processing (Colfer & Baldwin, 2016). For example, a platform team may embed with feature teams and learn multiple subsystems so it can spot system-wide tradeoffs rather than optimizing only one module.

Specifically, the *mirroring trap*, where "strictly mirrored firms perform poorly in the face of new technologies arising outside their boundaries" (Baldwin, 2024, p. 40).[2] The mirroring trap arises from failing to explore new knowledge. It can be avoided by adopting the partial mirroring strategy, which defines knowledge boundaries more broadly than task boundaries, even if it contradicts the concept of information hiding. For example, even if suppliers "own" a component, the buyer may still develop in-house expertise and prototype alternatives so it can absorb an external technological shift. This strategy was found to be effective in managing knowledge across 16 studies of buyer-supplier relations and of systems integrators who manage entire technical systems (Colfer & Baldwin, 2016).

Finally, misting the mirror has been shown to enhance information processing and promote coordination in complex situations (Billinger, Benincasa, Baumann, Kretschmer, & Schumacher, 2023; Brusoni, Prencipe, & Pavitt, 2001; Furlan et al., 2014). However, misting the mirror and rearranging dense technical interdependencies across organizational boundaries can yield radical and disruptive changes, providing a competitive advantage (Billinger et al., 2023). This approach is often observed in alliances and consortia, where multiple organizations collaborate to develop technically demanding new products or services. While mirroring is the predominant pattern in the economy, overly strict mirroring can be counterproductive (Baldwin, 2023; Fang & Kim, 2023).

Empirical studies provide confirmation of benefits of mirror misting. Parker and Anderson (2002) found that complex technical systems involving multiple companies require active coordination by a systems integrator, who needs diverse technical skills and knowledge beyond

[2] The trap argument is echoed in the strategic IT alignment (SITA) research. Sabherwal et al. (2019) cites prior arguments that strategic alignment "may sometimes reflect a rigidity, as implicit in prior arguments associating it with 'tunnel vision' (Tallon and Pinsonneault 2011), 'complacency and inertia' (Sabherwal et al. 2001), and a 'rigidity trap"' (Chen et al. 2010; Tallon 2008)." (p. 455).

their own tasks (see also, Brusoni et al., 2001; Constantinides, Henfridsson, & Parker, 2018). Similarly, Furlan et al. (2014) investigate how technological change affects the relationship between product modularity and organizational modularity, and the results show that the mirroring effect does not hold for technologically dynamic components. In such cases, the mirror gets "misted up," and buyers and suppliers engage in highly interactive ("thick") relationships even in the presence of highly modular components. Finally, Lee and Berente (2012) examined the impact of digital innovation on the division of innovative labor in the automotive industry. They found that, consistent with the mirroring hypothesis, after a technological shift in the inclusionary hierarchy, automaker OEMs and component suppliers focused more on architectural and component innovation, respectively. However, in line with mirror misting, due to the technological shift in the digital control hierarchy, automaker OEMs have increased their propensity for component innovation. In contrast, suppliers have increased their propensity for architectural innovation.

2.5 Mirroring in OSS: Absent or Not?

What constitutes an organization and a product differs depending on the setting. Organizations are systems of interacting agents, with identifiable boundaries and system-level goals, towards which agents contribute (Puranam, Alexy, & Reitzig, 2014). Teams, therefore, can be treated as organizations, albeit on a smaller scale. In the context of OSS development, the development team creates a software system. The technical structure of the system is its architecture, defined by how its components relate or depend on each other. *The organizational, or team, structure is determined by how the team members interact or collaborate within the context of their work*.

The body of work on mirroring suggests that organizations must carefully weigh their approach to mirroring and modular decomposition to optimize performance and adapt to technological change (Baldwin, 2023). But what if organizations cannot impose such design

rules? This situation will likely arise in organizational forms in which members self-select into tasks, such as in OSS development (Howison & Crowston, 2014; Raveendran, Puranam, & Warglien, 2022). As a result, mirroring in OSS is mostly emergent (i.e., resulting from developers' actions) rather than agentic (i.e., resulting from deliberate managers' decisions).

In software development, technical dependencies exist between code modules when one module invokes another module's functionality to perform its operations. For example, in a three-tier application, the user interface layer depends on the business logic layer to function, while the business logic layer depends on the data layer. These dependencies define the project's technical architecture.

Mirroring is about the correspondence between the technical and organizational structures. The technical structure is conceived in terms of dependencies among the product's modules. Specifically, code dependencies such as imports and function calls (MacCormack et al., 2006). The organizational structure is conceived in terms of collaboration dependencies among developers. Collaboration necessitates coordination between developers. Because coordination requires the provision of information (Puranam, Raveendran, & Knudsen, 2012), collaboration dependencies often lead to communication among developers to coordinate their actions. Finally, collaboration among developers manifests in their coding activities, where they work together on modules within the system.

Building on the previous example, Alice and Bob are two developers on the team. If they work on the same layer or are assigned the same task (e.g., developing the user interface), they will need to collaborate and communicate frequently to coordinate their development efforts. Suppose Alice and Bob work on different layers (e.g., Alice is a user interface developer and Bob is a data layer developer). In that case, they do not need to collaborate beyond setting up the

design rules, such as the interface between the backend and frontend (Baldwin & Clark, 2000), and addressing unresolved issues unanticipated at design time (Lindberg, Berente, Gaskin, & Lyytinen, 2016).

Relative to the broader work on mirroring, which typically studies the production of physical goods, a few studies examine mirroring between the organizational and technical structures in software projects, including OSS. When developers choose their own tasks, their coding tends to reflect their personal preferences. However, Colfer and Baldwin's (2016) systematic review of mirroring in OSS suggests much weaker evidence of mirroring in open collaboration projects. Such studies, primarily focused on software, have provided the least support for the mirroring hypothesis (p. 726). This weak evidence is surprising, given that OSS tends to be modular (MacCormack, Baldwin, & Rusnak, 2012), and OSS developers tend to specialize in different aspects of the project (Howison & Crowston, 2014; Medappa & Srivastava, 2019).

Colfer and Baldwin offer one explanation. Developers can work independently without direct coordination in a modular technical system if the components are easily understood and can be modified without affecting the entire system. The system provides a summary of its state, allowing team members to coordinate efforts by interacting with it. "When the individual components of a technical system can be comprehended easily and changed readily without propagating into the larger system, developers may not need to communicate directly with one another. Instead, the system itself summarizes its state, and interaction with the changing system suffices to coordinate the work of many independent agents" (Colfer & Baldwin, 2016, p. 727).

While the premise of the explanation seems reasonable, it does not necessarily preclude mirroring in OSS. Namely, if developers do not need to coordinate or communicate, the team structure also becomes modular as a consequence. A second explanation centers on the

availability of various communication channels. “If instantaneous high-bandwidth communication linkages are a substitute for colocation and a common employer, then we would expect to see more exceptions to the mirroring hypothesis in firms and projects supported by digital technology than traditional firms” (Baldwin, 2024, p. 38).

These two explanations may account for low or weak mirroring in OSS projects, but do not rule out the importance of mirroring. Indeed, it would be surprising if OSS projects did not accrue the benefits of mirroring when they adopt design rules similar to those of non-OSS projects and envision effective coordination mechanisms (MacCormack et al., 2012; Shaikh & Henfridsson, 2017). These two explanations provide a baseline for our study to depart from. Mirroring in OSS is feasible but challenging to maintain, given the lack of a central authority to enforce it as a design rule. Mirroring is brittle because many developers may be unaware of its existence or even its desirability. Therefore, it is only natural to find weak mirroring in OSS projects.

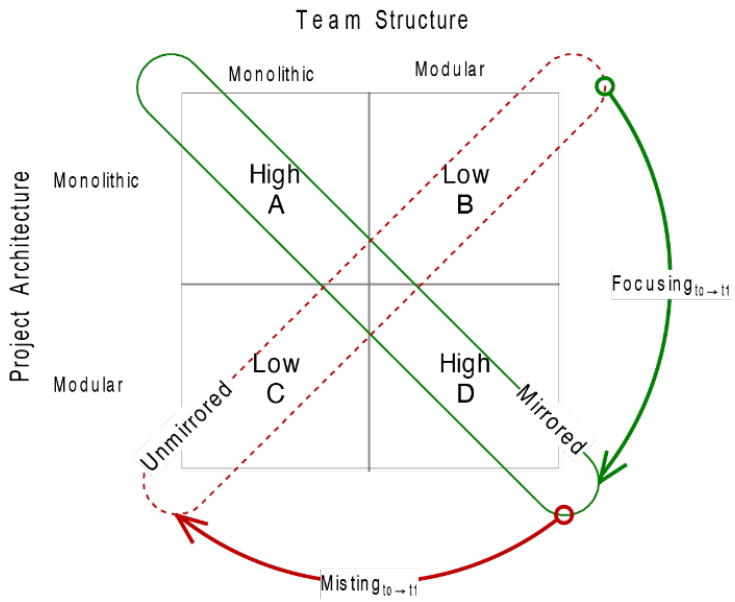
At the same time, because OSS projects are open and fluid (developers can join or leave at will), there are more opportunities for deviating from existing mirroring as new developers can effectively act as oblivious mirror misters, as well as reestablishing mirroring through modularization and specialization (Baldwin & Clark, 2006; Howison & Crowston, 2014). Therefore, when viewed as a state, mirroring in OSS is weak (Colfer & Baldwin, 2016). However, when viewed dynamically, mirror*ing* is prevalent in OSS. Specifically, while developers could inadvertently mist the mirror, development could incorporate many mirror-focusing activities, similar to those in software development and manufacturing projects.

2.6 Mirroring in OSS: State and Dynamics

OSS projects are fluid. Their codebase and development team composition change constantly over time. Such fluidity means that, while architecture is often conceived early in the software

development lifecycle (Booch, 1990), the development team may not always understand the implications of those decisions. Some developers may make software changes that violate the project's design rules. While OSS developers may not always intentionally commit design violations, the absence of a central decision-maker, such as a product architect, as a project evolves, increases the likelihood of such violations. Thus, when it comes to mirroring, it is more likely that a new activity deviates from established mirroring. New code added to a codebase may not align with the team structure, and vice versa. However, OSS code tends to be modular, and developers self-select and specialize in particular tasks (Howison & Crowston, 2014; Lerner & Tirole, 2002; MacCormack et al., 2012; Roberts, Hann, & Slaughter, 2006). As a result, these violations can be corrected as new developers get acclimated to the project and its codebase.

Similar to the capability view of IT-alignment (Sabherwal et al., 2019), our focus on mirroring dynamics emphasizes the importance of change over stability, particularly in complex environments. Mirroring dynamics, focusing and misting, as manifested in changes between mirroring states, could be as important as the states themselves. While changes between contrasting mirroring activities seem unproductive, they can lead to improvements (Nickerson & Zenger, 2002). Figure 1 shows that mirroring dynamics change the mirroring state either positively (mirror focusing) or negatively (mirror misting). These two dynamics could lead to monolithic and modular team and project structures.



Viewed as state, mirroring is the correspondence between project architecture and team structure. Adopting a simplified 2x2 classification, it has four archetypes:

- Two high mirroring: A and D
- Two low mirroring: B and C

Viewed dynamically, mirror*ing* is the intertemporal change in the correspondence between project architecture and team structure. With the previous classification, it has eight archetypes that describe all possible dynamics:

- Four mirror-focusing: B→A, C→A, B→D, and C→D
- Four mirror-misting: A→B, D→B, A→C, D→C

Figure 1: Archetypes of mirroring from a state and dynamics perspectives

2.7 Propositions

Building on prior work, because of their open membership and fluid boundaries, OSS projects exhibit weak mirroring (Colfer & Baldwin, 2016). Developers mist the mirror when actively experimenting, exploring, and adjusting the design *across* components (e.g., modules in a larger codebase). Some developers may decide to postpone critical development activities (Howison & Crowston, 2014), leaving others to fill in the gaps. In collaborative undertakings, tinkering can lead to a deeper understanding by enhancing comprehension through the interrelation of multiple levels of experience and connecting newcomers with seasoned professionals (Weick & Roberts, 1993). When it comes to OSS, developers learn about the project by tinkering with it (G. K. Lee & Cole, 2003; Singh, Tan, & Youn, 2011). However, as developers gain a deeper understanding of the project architecture, they can improve coordination by reducing overt coordination costs, decreasing cross-module integration costs, and decreasing the need for knowledge outside the task boundaries of module developers (Tiwana et al., 2010). These activities enable them to restructure their work and the project architecture around each other and, in effect, refocus the

mirror. Because of the ebbs and flows of project membership and coding activities, we propose that:

Proposition 1: Mirroring dynamics fluctuate in OSS projects. Specifically, OSS projects alternate between mirror misting and mirror focusing.

As discussed in §2.4, the normative view of mirroring holds that mirroring is advantageous, and therefore establishing it (i.e., mirror focusing) brings benefits. On the other hand, several studies demonstrate the benefits of mirror misting, mainly through knowledge spillover, where drawing organizational boundaries beyond technical ones creates opportunities for broader knowledge sharing. A dynamic view of mirroring could integrate these two observations. Mirror misting creates the opportunity to learn about aspects of the projects beyond one's specialization. From an evolutionary standpoint, this new knowledge could be leveraged in future iterations by applying it to specific modules through mirror focusing. For example, when the platform team applies knowledge of specific features to redesign the platform.

In OSS, mirror focusing is emergent rather than strategically planned. Unlike organizations where managers deliberately adjust structures, OSS projects lack centralized authority to impose design rules. Misting arises from distributed, decentralized developer actions. Some inadvertently deviate from established mirroring; others restore them through modularization. Change thus reflects system-level properties of fluid, open collaboration rather than deliberate management.

Specifically, when developers tinker by misting and then refocusing the mirror, they experiment with how modules interact, assess where dependencies bottleneck, and consider which interfaces can be redesigned or recombined. This experimentation builds architectural knowledge (how the system is structured) and meta-knowledge (who knows what and how work

is coordinated), improving the fit between developers' skills and development tasks, and accelerating both specialization and integration work (Andersson, Lindgren, & Henfridsson, 2008; Sosa, Eppinger, & Rowles, 2004). These knowledge gains have positive effects on project evolution because they enable developers to reconfigure modules more confidently, introduce or retire features, and adjust component properties (e.g., performance, reliability, maintainability) in response to emerging requirements—thereby producing changes in project functionality or properties. Moreover, because evolution is path-dependent (Lehman, 1979; Tiwana et al., 2010), the coordination improvements generated by mirror focusing compound over successive development cycles. When developers realign team collaboration patterns to match the project's technical architecture, they reduce friction in cross-module integration and lower the cognitive overhead of navigating unfamiliar code. These coordination efficiencies free resources for substantive development work such as implementing new functionality, refactoring aging components, and extending interfaces. Such activities directly manifest as project evolution. In addition, the knowledge recombination that occurs during mirror focusing enables developers to apply insights gained during the misting phase to their specialized modules, translating cross-boundary exploration into localized improvements that accumulate as evolutionary change (Colfer & Baldwin, 2016; Windrum et al., 2019). We therefore propose that:

Proposition 2: Mirror focusing promotes project evolution.

Implicit in our previous arguments is that mirroring dynamics lead to evolutionary outcomes when they enable developers to mist the mirror and subsequently refocus it. The benefits accrued during the misting stage include learning about other modules (Baldwin, 2024). These benefits materialize when developers refocus the mirror, specializing in their modules. Therefore, mirror misting followed by mirror focusing is beneficial when it leads to refocusing the mirror to a

modular-modular configuration, as we further argue here. This process is therefore akin to the entrepreneurial activities of creative destruction in economics (Schumpeter, 1942) or antifragile tinkering in complex systems (Taleb, 2012). Re-focusing the mirror can arise in two conditions: monolithicity and modularity of project and team structures (A and D in Figure 1).

The definition of mirroring as the correspondence between system architecture and team structure suggests that unmirrored or weakly mirrored systems can focus their mirroring by becoming more modular or more monolithic. Consider quadrants B and C, where the system is weakly mirrored. Such a system can enhance its mirroring through *modularization* (B→D and C→D). An alternative path is to enhance mirroring through *monolithization* (B→A and C→A).

Interestingly, while prior work largely agrees upon the definition of mirroring as the correspondence between project and team structure, they implicitly examine mirroring under a modular configuration (D) when a modular organization develops a modular product and not the other way around (Burton & Galvin, 2022; Chen et al., 2019; Hao et al., 2017; Sorkun & Furlan, 2017). There are good conceptual reasons to exclude mirroring under monolithic team and project structures, given a wealth of arguments for modularity (Baldwin & Clark, 2000). Because they are costly to maintain, modularity as a design rule aims to minimize code dependencies. In our previous example, we expect developers to specialize in the three modules (model, view, and controller) rather than a different division of labor. Some studies have highlighted the advantages of monolithic architecture over modular architectures in terms of performance (Tiwana, 2018). Notwithstanding such nascent evidence, given the consensus on modular mirroring, we propose:

Proposition 3: Mirror focusing promotes project evolution when it re-focuses the mirror into a modular state (B→D and C→D).

We have developed three broad propositions. We expect OSS projects to exemplify mirroring dynamics in terms of the prevalence of mirror misting and mirror focusing, and we expect these processes to be important in OSS project evolution. We later discuss some of the boundary conditions that could inhibit these dynamics.

3 METHODS

We have conceived a comprehensive archival data approach to examine a large sample of evolving OSS projects. Figure 2 summarizes the methodological flow from data collection to inference. It illustrates the progression from sampling and quarterly reconstruction of OSS projects, through measurement of mirroring constructs and outcomes, to analytical models.

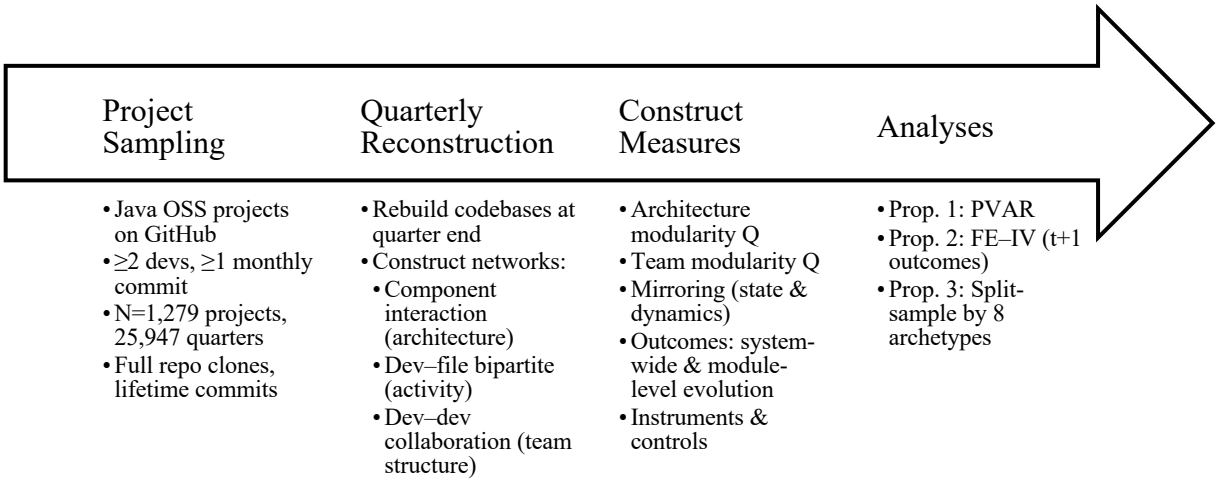


Figure 2: Methods and analyses

3.1 Context and Sampling Strategy

As part of a larger research program on IS evolution, we used a random sample of 1,279 open-source Java projects. We focused on Java for three reasons. First, focusing on a single programming language mitigates that could otherwise confound our results. Second, Java is a single-paradigm language that uses object-oriented programming (OOP). This property enables us to compare projects using consistent architectural measurements across diverse projects and time periods. Third, Java is used across a broad spectrum of domains for developing mobile,

desktop, and enterprise applications; it has remained among the top three programming languages by market share during the last fifteen years (TIOBE, 2020). This popularity ensures sufficient variance, mitigates confounding due to the idiosyncrasies of one domain, and broadens the generalizability of our findings.

We sampled from GitHub to mitigate confounding due to differences in software configuration management, lifecycle maintenance, and source code versioning tools. GitHub is the most popular open-source repository, hosting many open-source projects. At the time of sampling, 24 million users spanning 1.5 million organizations hosted 67 million code repositories on it. It spans diverse projects, including professional teams, hobbyists, community-driven, and firm-driven commercial projects. Unlike rivals like SourceForge, GitHub provides a unified version control system (Git) and default configuration management services such as issue tracking, documentation, and project management, eliminating confusion caused by the choice of OSS development tools. The projects in our sample span a wide range of categories (Figure 3).[3]

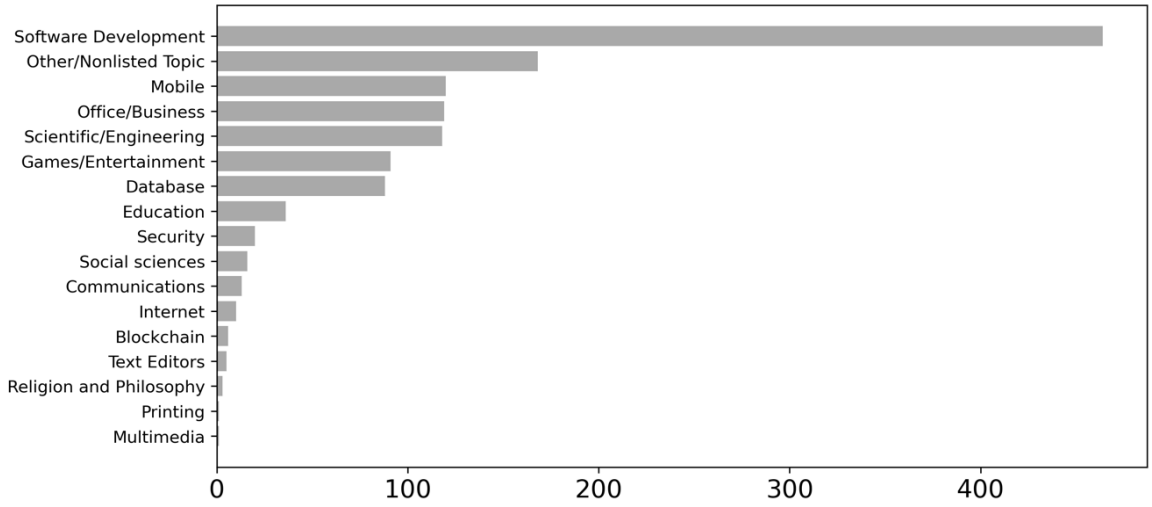


Figure 3: Categories of the sampled projects

[3] GitHub does not provide a project category. The first author and a research assistant each coded the projects using the categories offered by SourceForge https://sourceforge.net/directory/ and reconciled any disagreements.

3.2 Data Collection and Provenance

As part of a broader research program to study long-run patterns in software evolution (citation blinded), we generated the sampling frame using GHTorrent, a comprehensive database of GitHub projects (Gousios & Spinellis, 2012), while following the established best practices for project selection (Kalliamvakou et al., 2016).

Our population was OSS projects that were actively under development. To ensure meaningful testing of the proposed ideas, we required at least two distinct developers (since single-developer projects lack team structure) and sufficient consecutive quarters of activity to observe temporal dynamics in substantive collaborative development.[4] Specifically, we focused on projects with at least one monthly commit between 2011 and 2016, excluding projects forked from other projects. We then locally cloned the source code for each project and collated its lifetime commits. Each code commit includes an identifier, a timestamp, the developer's username, the changed files, and the committed code. Each project was observed from its inception until the end of 2016. We stopped in 2016 due to a major change in Java that prevents comparability with prior code.[5]

The average lifespan of a project in the sample is 6 years. Using this fine-grained data, we organized our data into a large panel. We reconstructed the state of each project at every quarter-end by accumulating commits submitted up to that point. We also reconstructed development activity data by using commit data to link developers to changed source code files, enabling us to examine the evolution of each project by comparing its codebase across time periods. We used a quarter as the observation interval following Zaheer and Zaheer's (1999) logic to use an interval

[4] In some project quarters, one developer was making a single commit. However, across their lifetimes in our observation window (2011-2016), all projects had at least two distinct developers.

[5] Java 9 introduced the new concept of "module" which allows for further grouping and bundling of packages. Since the code in this study predates this addition, we can safely consider a package to be the main modular unit.

long enough for changes to be observable but not too long for confounds to enter. We considered a monthly interval but rejected it because many projects had only one commit per month (37%), making it difficult to observe changes from a single commit.

3.3 Measures

We used a comprehensive measurement approach to operationalize the research constructs using the commit logs of OSS projects (Table 1 provides conceptual and operational definitions and Table 2 shows descriptive statistics and correlations). First, we operationalized the project architecture and team structure needed to measure mirroring. In particular, developers' activity defines a project's architecture and the team structure that produced it. Architecture refers to "(1) the arrangement of functional elements; (2) the mapping from functional elements to physical components; (3) the specification of the interfaces among interacting physical components" (Ulrich, 1995, p. 420). This definition aligns with software components and their interactions in OSS projects. In an object-oriented language such as Java, components are classes, and their interaction takes one of two forms: composition or inheritance (Booch, 1990). Similarly, the team structure refers to how team members interact to achieve their collective goals. It involves the arrangement of team members, the division of labor, and the establishment of communication and coordination mechanisms (Monge & Contractor, 2001; Murase, Carter, DeChurch, & Marks, 2014; Srikanth & Puranam, 2011).

Table 1: Key theoretical constructs and their measurements (all at the project/quarter level of analysis)

Construct		Description
		Mirroring-related
Project architecture	*Definition*	*How components in the product interact to realize its function*
	Measurement	Modularity Q of the class interaction network
Team structure	*Definition*	*How team members interact to achieve their collective goals*
	Measurement	Modularity Q of the developer collaboration network
Mirroring state	*Definition*	*The correspondence between the project architecture and the team structure*
	Measurement	$1 - \frac{\lvert project\ achitecture\ modularity\ -\ team\ structure\ modularity \rvert}{project\ achitecture\ modularity\ +\ team\ structure\ modularity}$
Mirror focusing	*Definition*	*The (positive) intertemporal change of correspondence between the project architecture and team structure*
	Measurement	$\Delta_{t-1 \to t} Mirroring\ state = Mirroring\ state_t\ -\ Mirroring\ state_{t-1}$
		Outcomes
System-wide evolution	*Definition*	*Changes of the components of the project since the last time period*
	Measurement	$1 - \frac{\lvert Files_t \cap Files_{t-1} \rvert}{\lvert Files_t \cup Files_{t-1} \rvert}$
Module-level evolution	*Definition*	*Changes within the components of the project since the last time period*
	Measurement	$1\ -\ \frac{\lvert Files\ unchanged\ (same\ source\ code)_{t,t-1} \rvert}{\lvert Files_t \cup Files_{t-1} \rvert}$
		Instruments
Work specialization	*Definition*	*The extent to which developers specialize in different components*
	Measurement	The average length of shortest paths in the development activity network
Work centralization	*Definition*	*The extent to which few developers do most of the work*
	Measurement	Freeman centralization of the development activity network
		Control variables
Project age	*Definition*	*The age of the project*
	Measurement	Number of years since the project's inception
Team size	*Definition*	*The size of the project development team*
	Measurement	The number of developers committing to the project during t-1 and t
Project size	*Definition*	*The size of the project codebase*
	Measurement	The number of source code lines (in millions)
External dependencies	*Definition*	*The project's reliance on external dependencies*
	Measurement	Log(1 + number of external packages imported to the project)

Previous research on product architecture utilizes the concept of a design structure matrix (DSM), which we build on. A DSM is a visual tool that depicts the interdependencies and relationships among elements or components in a design. It is represented as a square matrix, with rows and columns corresponding to the design elements. The cells in the matrix indicate the dependency relations between the elements, showing which elements are structurally dependent on each other or have been changed together. The DSM helps highlight a design's inherent structure and identify the level of coupling or modularity between different parts of the system (Baldwin & Clark, 2000; MacCormack et al., 2012; Sosa et al., 2004). Effectively, a DSM is a network in which nodes represent design elements and edges represent their dependencies.

We implement the structure matrix within the context of software projects and expand it to represent not only the project architecture but also the dynamic team structure as reflected in the team's actual development activities. We use networks to map the interaction between interdependent elements (components in the case of system architecture and developers in the case of team structure). Specifically, using commit logs, we construct three networks (Figure 4).

The first network (a) is the component interaction network created by analyzing the source code of each project in each quarter. This network's nodes correspond to classes and interfaces, and their interactions are composition and inheritance relationships. Next, we examine developers' activity in each quarter. These actions are committing code to (some) files in the project, creating, as a result, a bipartite network (b) of two sets of nodes (developers and files). When two developers commit to the same file during the same quarter, they effectively collaborate on evolving it. Thus, a third network of developer collaboration (c) is created by connecting developers who commit to the same files in the same quarter. Collaboration is evidenced by two developers co-changing the file in the same time period.

Networks (a) and (c) represent the project architecture and team structure during an observation period. A variety of metrics exist to operationalize network structure. Given that modularity is the most important architectural property in software and OSS development (Baldwin & Clark, 2000; MacCormack et al., 2012), we use network modularity (Q) to measure it in these networks (Newman, 2006; Newman & Girvan, 2004).[6] This measure has the benefit of

[6] Modularity *Q* refers to the measure of how well a network can be divided into distinct communities or modules. It quantifies the arrangement of edges within groups compared to what would be expected in a randomly connected network. A higher modularity value indicates a stronger presence of community structure within the network. Modularity Q is calculated using a mathematical formula that involves the evaluation of the modularity matrix and the leading eigenvector. The algorithm used to determine the modularity is able to effectively divide networks into communities but also has the ability to recognize when a network is indivisible, meaning it cannot be further divided. The algorithm aims to maximize modularity by concentrating as much weight as possible in the sum involving the largest eigenvalues. The modularity Q values provide a quantitative measure of the algorithm's success in identifying community structure in real-world networks (Newman, 2006).

a range from 0 (fully monolithic) to 1 (fully modular), making it agnostic to network size and useful for assessing the correspondence of networks of varying sizes.[7]

Figure 5 shows two project-quarter observations with the same high mirroring value of 0.99, albeit with higher (top panel) and lower (bottom panel) values for project architecture and team structure Q. In the figure, the circle nodes represent classes, and the square nodes represent developers. Gray arrows represent class dependencies, and black arrows represent developer commits to source code files defining these classes. Both examples show clusters of classes of high interdependencies. The top project-quarter shows fewer inter-cluster dependencies (hence a higher modularity q for the class interaction network) and fewer developer co-commits across clusters (thus a higher modularity q for the developer co-commit network).

Next, building on the definition of mirroring as the correspondence between the project architecture and team structure (Colfer & Baldwin, 2016), we operationalize this correspondence as the normalized distance between the two. Since mirroring is about correspondence and closeness, we invert the distance measure to measure mirroring as a state. Mirroring dynamics involve changing this state, either by increasing (i.e., focusing) or decreasing (i.e., misting) mirroring as a state. Our operationalization captures change *magnitude* rather than oscillation *frequency*. This magnitude-based Δ-mirroring measure indicates shifts in architecture-team correspondence through focusing (positive Δ) or misting (negative Δ). While frequency measures would more directly capture vacillation, quarterly observations provide a tractable proxy; the negative autocorrelation in Table 3 confirms that these magnitude changes reflect such dynamics. Thus, we operationalize mirror focusing as the difference between the state of mirroring since the last time period. A negative value of mirror focusing corresponds to misting.

[7] Naturally, the component interaction network is much larger than the developer coordination networks, as there are many more components in a project than developers.

Next, building on the distinction between within-component change and across-component change, we operationalize the two system evolution constructs, leveraging Henderson and Clark's (1990) two dimensions of change. System-wide evolution refers to changes of the components of the system, and module-level evolution refers to changes within the modules of the project. Because Java source code files implement OOP components (classes and interfaces), we measure system-wide evolution as the inverse Jaccard set similarity between project files in the current and previous quarters. Specifically, we count the number of files added or removed from the codebase and normalize it over the total number of files. We measure module-level evolution by tracking the content of project files that have persisted since the last quarter. Specifically, files that were neither added nor removed can remain unchanged in the source code or be changed when developers modify them. Since these modifications are internal to the files and the components, they measure module-level evolution.

We measure the following variables and use them as instruments or control variables. Work centralization refers to the extent to which a small number of developers perform the majority of the work. We measure it using the Freeman centralization statistic (Freeman, 1979), which calculates the sum of differences between the (closeness) centrality of the most central node and all other nodes, divided by the same sum calculated on a star graph (most centralized graph) with the same number of nodes (Borgatti & Everett, 2006). Work specialization refers to the extent to which developers specialize in different components. A strict division of labor exists when developers take ownership of distinct files in the project (no file is committed to by more than one developer). However, such extreme division of labor is rare in OSS, where developers build on each other's work (Howison & Crowston, 2014). We measure specialization by calculating the average length of shortest paths within the development activity network. When developers

collaborate on multiple files, this length is reduced (resulting in less specialization) and vice versa. Project age is the number of years since the project's inception. Team size refers to the number of developers committed to the project during the specified time period. Project size refers to the number of source code lines, measured in millions. Finally, the logged count of imported external packages in the project reflects the project's external dependencies.

Table 2: Descriptive statistics and correlation matrix, 25,947 quarterly observations of 1,279 projects, * p <.05

	mean	sd	min	p25	p50	p75	max
(1) Project architecture	.57	.12	0	.50	.57	.65	.97
(2) Team structure	.058	.10	0	0	0	.080	.85
(3) Mirroring state	.14	.22	0	0	0	.25	1.00
(4) Mirror focusing	.0073	.22	-1.00	-.015	0	.052	1.00
(5) System-wide evolution	.15	.22	0	.015	.062	.18	1
(6) Module-level evolution	.36	.32	0	.094	.26	.56	1
(7) Work specialization	2.38	.66	.50	1.98	2.18	2.76	10.8
(8) Work centralization	.42	.16	0	.30	.46	.50	.84
(9) Project age	3.27	2.88	0	1.25	2.50	4.50	18.2
(10) Team size	5.62	6.15	1	1	4	7	113
(11) Project size	6.56	11.2	.000094	1.08	2.86	7.30	176.1
(12) External dependencies	4.51	1.10	0	4.03	4.62	5.21	6.99

	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)	(11)	(12)
(1)	1	.21*	.14*	.0022	-.005	-.11*	.24*	.1*	.1*	.15*	.17*	.42*
(2)	.21*	1	.97*	.49*	-.09*	-.15*	.5*	.3*	.16*	.52*	.26*	.34*
(3)	.14*	.97*	1	.5*	-.094*	-.13*	.53*	.29*	.17*	.57*	.26*	.34*
(4)	.0022	.49*	.5*	1	.021*	.0022	.13*	.018*	-.02*	.11*	-.0046	.0027
(5)	-.005	-.09*	-.094*	.021*	1	.76*	-.046*	-.056*	-.27*	.035*	-.17*	-.24*
(6)	-.11*	-.15*	-.13*	.0022	.76*	1	-.088*	-.024*	-.33*	.11*	-.2*	-.23*
(7)	.24*	.5*	.53*	.13*	-.046*	-.088*	1	.18*	.14*	.6*	.32*	.42*
(8)	.1*	.3*	.29*	.018*	-.056*	-.024*	.18*	1	.12*	.14*	.17*	.17*
(9)	.1*	.16*	.17*	-.02*	-.27*	-.33*	.14*	.12*	1	.085*	.27*	.31*
(10)	.15*	.52*	.57*	.11*	.035*	.11*	.6*	.14*	.085*	1	.34*	.37*
(11)	.17*	.26*	.26*	-.0046	-.17*	-.2*	.32*	.17*	.27*	.34*	1	.46*
(12)	.42*	.34*	.34*	.0027	-.24*	-.23*	.42*	.17*	.31*	.37*	.46*	1

The three networks are constructed from the commit logs of the STEP project in Q2 of 2014.
"STEP is a free, high-quality, portable web-based, online and offline Bible study tool comprising many versions across hundreds of languages, with many different viewing options to ease personal study" (https://api.github.com/repos/tyndale/step)

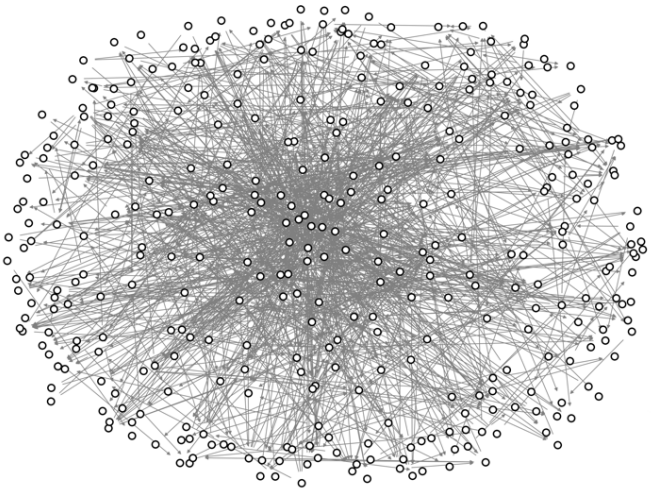
The component (classes and interfaces) interaction network defines the project architecture (a). In particular, 361 components (○) and 1244 interactions (→) reflect composition and inheritance relationships across these components.

During Q2 of 2014, four developers (■) contributed to 142 files (○) through 186 code commits (→). This is reflected in the development activity network (b).

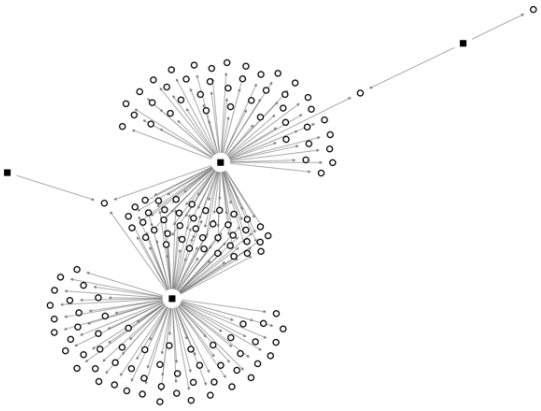
Examining the division of labor reveals that two developers (■) collaborated (↔) extensively on some core files while two peripheral developers contributed to three other files. The two core developers also contributed to some other files. These collaborations are reflected in the network (c), which models the team structure.

Networks

(a) Project architecture (component interaction network)



(b) Development activity (developers change some project files)



(c) Team structure (developer collaboration network)

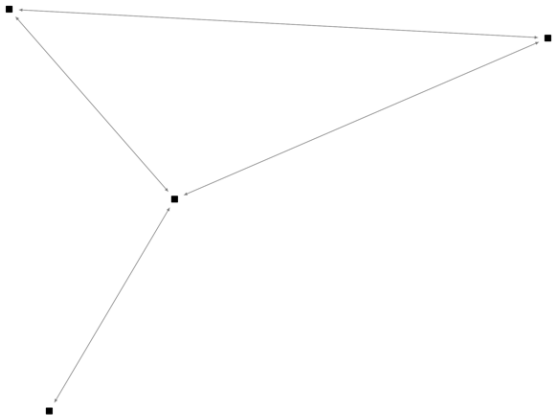


Figure 4: Examples of interaction networks used to measure project architecture, team structure, and mirroring for one illustrative project.

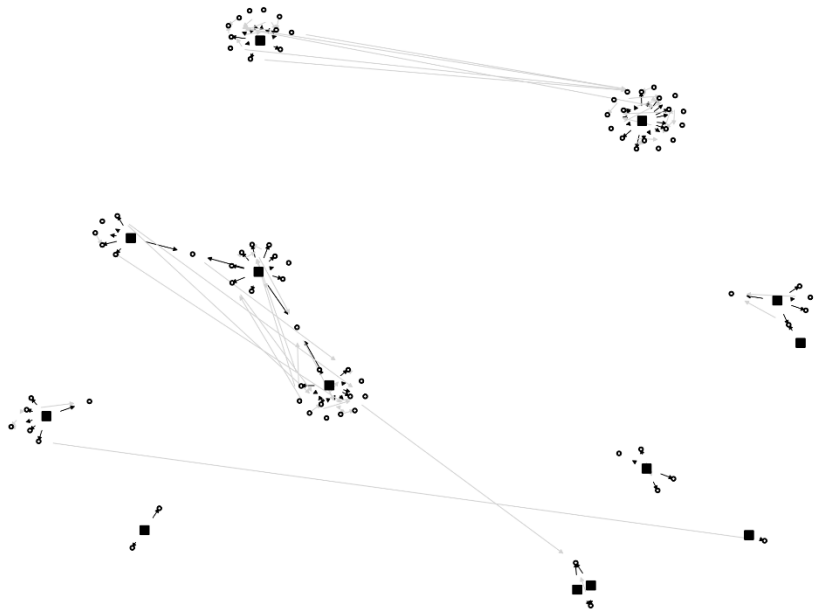
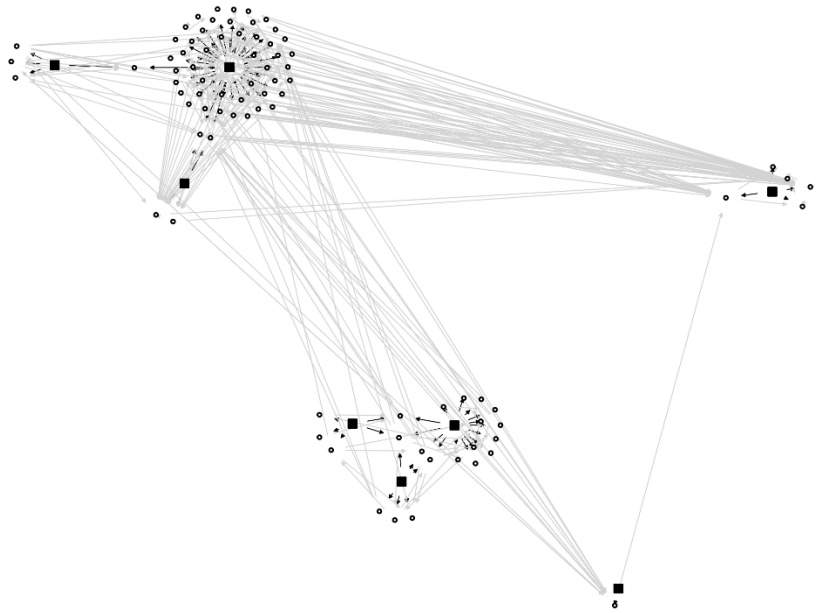
Project-quarter	Developer commits and module interactions
community-edition in 2013Q4 Mirroring state = 0.99 High class interaction network modularity q = 0.613 High developer collaboration modularity q = 0.625	
Hystrix in 2016Q3 Mirroring state = 0.99 Low class interaction network modularity q = 0.48 Low developer collaboration modularity q = 0.48	

Figure 5: Two project-quarters of the same high value of mirroring, demonstrating mirroring under high (top) and low (bottom) project architecture and team structure modularity, respectively

3.4 Modeling Approach

To test the first proposition, we are interested in changes in mirroring dynamics over time. We can extract visual evidence by plotting the time series values (Figure 6). However, a statistical test rests on testing the lagged effects of the dependent variable on its current values (i.e., mirror focusing$_{t-1}$→mirror focusing$_t$). A negative effect supports the proposition that focusing and

misting alternate. However, using the lagged dependent variable as an independent variable violates assumption 4 of the classical linear regression model and can result in asymptotic bias (Kennedy, 2008, p. 160). However, it can be a viable instrument for the lagged dependent variable in a simultaneous equation system (Roodman, 2009). A panel vector autoregression (PVAR) model is a statistical model used in applied research that combines time-series VAR models with panel data. It is used to analyze the relationships and dynamics among multiple variables over time, accounting for both the individual characteristics of each panel unit and the time-series properties of the data (Abrigo & Love, 2016).

This advanced econometric approach helps address endogeneity and produces consistent, robust results. Additionally, the panel VAR framework considers lagged interdependences and fixed effects, allowing for the analysis of relationships across multiple interdependent variables while accounting for potential heterogeneity (Abrigo & Love, 2016; Keane & Runkle, 1992). PVAR has been recently used to study the evolution of complex systems where many variables are endogenous (Acheampong, 2018; Foster, Grim, Haltiwanger, & Wolf, 2021). For instance, studying the relationship between the generativity of digital platforms and the growth of their user base, Fürstenau et al. (2023) used the PVAR model to estimate the coefficients of interdependencies among generativity, social interaction, and user-base growth variables.

In this study, we use PVAR to estimate the effect of mirror focusing$_{t-1}$ on mirror focusing$_{t}$. We also perform the same test for mirroring as a state to contrast the two views on mirroring introduced here. In both estimations, we control for project size and external dependencies as endogenous variables and project age and team size as exogenous variables. The reason for treating project size and external dependencies as endogenous is that they are affected by the development process (developers write more code or decide to import external code).

PVAR model

$$\text{Mirroring state}_t = \beta_0 + \beta_1 \text{Mirroring state}_{t-1} + \beta_2 \text{Project age}_t + \beta_3 \text{Team size}_t + \beta_4 \text{Project size}_{t-1} + \beta_5 \text{External dependencies}_{t-1} + \varepsilon$$

$$\text{Mirror focusing}_t = \beta_0 + \beta_1 \text{Mirror focusing}_{t-1} + \beta_2 \text{Project age}_t + \beta_3 \text{Team size}_t + \beta_4 \text{Project size}_{t-1} + \beta_5 \text{External dependencies}_{t-1} + \varepsilon$$

To test the second proposition, we perform a regression analysis to examine the effect of mirroring dynamics on project evolution (both system-wide and module-level) while controlling for the mirroring state and other control variables. If any factors that could affect a project's evolution are not considered, the OLS estimates may be biased. Unobserved predictors are likely to be project and team characteristics that remain consistent over time, as well as those that change and can impact development. For example, consider the following factors when evaluating a project: availability of software components or microservices in the domain, involvement of a renowned architect (e.g., Linux), sponsorship by a major company (e.g., Android), or a history of prior collaborations among individuals involved in the project.

We employed fixed-effects regression to account for unobserved differences in our unbalanced multi-period panel data (Allison, 2009). By entering dummies for each of the 1,279 projects, this approach accounts for time-invariant, project-specific omitted variables in our analyses. Each project serves as its own control, accounting for all stable, unobserved variables as if they had been included in the regression model. This strategy serves the same function as a random assignment in an experiment (Allison, 2009, p. ix). A fixed-effects model also mitigates selection effects; i.e., projects might enter or exit the panel because of particular attributes and therefore do not represent a truly random sample.

A fixed-effects model exploits the fact that our panel has (a) multiple observations for every project and (b) observations from the same project are more similar to each other over time than from other projects. The identifying assumptions are: (a) that the omitted variables are *time-invariant* and (b) that there is enough variance within apps in our core theoretical variables.

Following Kennedy's (1992, p. 223) guidelines, we used the Durbin–Wu–Hausman test to assess if random-effects models were more appropriate than fixed-effects models. It rejected the null that the preferred model is random-effects ($\chi^2_{\text{system-wide}}$ 647; $\chi^2_{\text{module-level}}$ 209; $p<.001$), suggesting the appropriateness of fixed-effects models (Allison, 2009, p. 23). We found evidence for autocorrelation of error terms because the Breusch-Pagan (1979) test's null of no serial correlation was rejected for both dependent variables ($F_{\text{system-wide}} = 35.9$; $F_{\text{module-level}} = 104.5$; $p<.001$) (Drukker, 2003). Further, heteroskedasticity is present (the modified Wald statistic for within-group heteroskedasticity in fixed-effect models rejected the null of homoskedasticity in both models ($\chi^2_{\text{system-wide}}$ 4.5e9; $\chi^2_{\text{module-level}}$ 8.5e10; both $p<.001$). Thus, we used heteroskedastic, cluster- and autocorrelation-robust standard errors (Arellano, 1987) implemented in the Stata xtivreg2 command (Schaffer, 2015). Additional tests provided assurance against multicollinearity (the highest variance inflation factor in our models was 2.17<5).

Including fixed effects, therefore, removes all time-*invariant* between-project variation. However, we must separately deal with time-*variant* predictors of evolution and project and project team architecture since fixed effects do not alleviate such endogeneity. We thus use a two-stage instrumental-variables approach to address this issue. We specifically use lagged values of work specialization and work centralization as instruments for mirroring.

Stage 1: $\text{Mirroring state}_t = \alpha_0 + \alpha_1\text{Work specialization}_{t-1} + \alpha_2\text{Work centralization}_{t-1} + \alpha_3\text{Project age}_t + \alpha_4\text{Team size}_t + \alpha_5\text{Project size}_t + \alpha_6\text{External dependencies}_t + \varepsilon$

Stage 2: $\text{System-wide evolution}_{t+1} \mid \text{Module-level evolution}_{t+1} = \beta_0 + \beta_1\text{Mirrorinng state}_t + \beta_2\text{Mirror focusing}_t + \beta_3\text{Project age}_t + \beta_4\text{Team size}_t + \beta_5\text{Project size}_t + \beta_6\text{External dependencies}_t + \varepsilon$

Good instruments should satisfy the criteria of independence, exclusion restriction, and variation and be chosen based on domain knowledge and ideas about the process determining the variable of interest (Angrist & Pischke, 2008, Chapter 4). The instrument should be independent

of potential outcomes (independence) and not affect outcomes other than through the first-stage channel (exclusion restriction). The instrumental variable should also help partially explain the variation in the endogenous explanatory variable (variation).

Within the context of OSS project development, developers' self-selection into development tasks in the previous time period sets the stage for development in the next time period. Thus, they affect the architectural outcomes, the emergent collaborative relations, and as a result, the mirroring between the project architecture and the team structure. However, these (lagged) activities are independent of current activities that shape project evolution. Therefore, the previous configuration of work specialization and work centralization only affects the current project evolution by affecting mirroring in the current period.

Empirically, appropriate instruments must be correlated with the endogenous predictors (the *relevance criterion*) but not with the error term of the dependent variables (the *exclusion restriction*). The correlation between mirroring state$_t$ and with work specialization$_{t-1}$ and work centralization$_{t-1}$ is 0.41*** and 0.25*** respectively. The correlation between system-wide$_{t+1}$ with work specialization$_{t-1}$ and work centralization$_{t-1}$ is -0.07*** and -0.08***, respectively. Finally, the correlation between module-level evolution$_{t+1}$ with work specialization$_{t-1}$ and work centralization$_{t-1}$ is -0.08*** and -0.06*** respectively. We further perform various statistical instrument validity tests. The results of the first stage regression are shown in Table 4. The large F-value (F=366>10) of our Stage 1 model provides preliminary evidence against instrument weakness. The F-test of excluded instruments (F=60.22) exceeded the Stock-Yogo (2002) 10% threshold of 19.93. The instruments rejected Kleibergen-Paap's (2006) test (χ^2=114; p<.001). The Stock-Wright (2012) Lagrange Multiplier (LM) test rejected the null of exogeneity and confirmed instrument sufficiency for both system-wide evolution ($\chi^2_{LM\text{-}S}$=10.11; p<.01) and

module-level evolution ($\chi^2_{LM\text{-}S}$=15.15; p<.001) with the null of under-identification. Finally, the Sargan-Hansen J-test, whose null is that the over-identifying restrictions are valid, was nonsignificant for both models ($\chi^2_{\text{system-wide evolution}}$ 3.7; $\chi^2_{\text{module-level evolution}}$ 2.6; both n.s.). Collectively, these tests suggest that our models are appropriately overidentified, and the instruments are valid.

Finally, to test the third proposition, we build upon the previous specification and we conduct a split-sample analysis by dividing the sample into eight subsamples based on the relative change or stability of $\Delta_{t\text{-}1\rightarrow t}$Project architecture and $\Delta_{t\text{-}1\rightarrow t}$Team structure. These subsamples correspond to all Low→High and High→Low changes in mirroring as a state (Figure 1):

- B→D: $\Delta_{t\text{-}1\rightarrow t}$Team structure >= 0 and $\Delta_{t\text{-}1\rightarrow t}$Project architecture > 0
- C→D: $\Delta_{t\text{-}1\rightarrow t}$Project architecture >= 0 and $\Delta_{t\text{-}1\rightarrow t}$Team structure > 0
- B→A: $\Delta_{t\text{-}1\rightarrow t}$Project architecture <= 0 and $\Delta_{t\text{-}1\rightarrow t}$Team structure < 0
- C→A: $\Delta_{t\text{-}1\rightarrow t}$Team structure <= 0 and $\Delta_{t\text{-}1\rightarrow t}$Project architecture < 0
- A→B: $\Delta_{t\text{-}1\rightarrow t}$Project architecture <= 0 and $\Delta_{t\text{-}1\rightarrow t}$Team structure > 0
- A→C: $\Delta_{t\text{-}1\rightarrow t}$Team structure <= 0 and $\Delta_{t\text{-}1\rightarrow t}$Project architecture > 0
- D→B: $\Delta_{t\text{-}1\rightarrow t}$Team structure >= 0 and $\Delta_{t\text{-}1\rightarrow t}$Project architecture < 0
- D→C: $\Delta_{t\text{-}1\rightarrow t}$Project architecture >= 0 and $\Delta_{t\text{-}1\rightarrow t}$Team structure < 0

4 RESULTS

We first seek to test the propositions. Then, further develop our understanding of mirroring dynamics, building on the initial results.

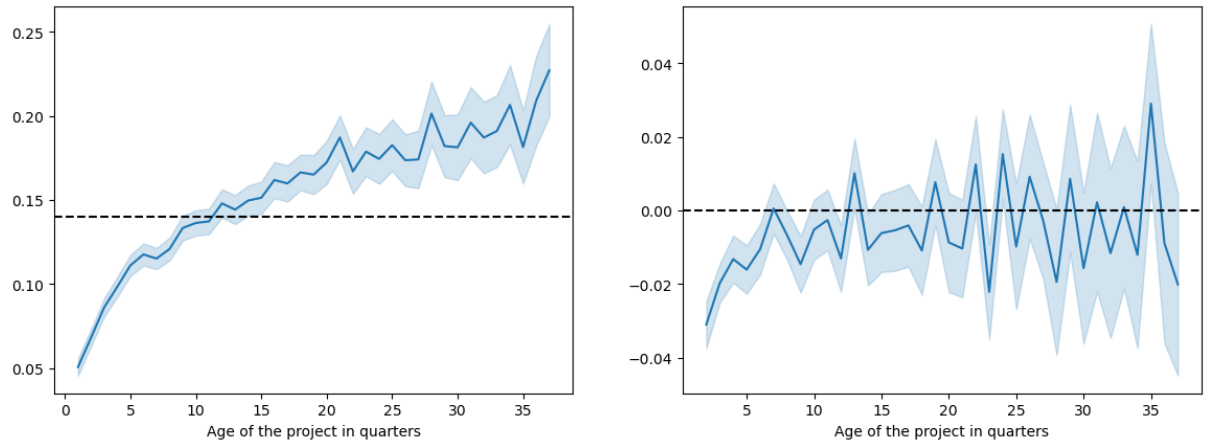


Figure 6: The Average value of mirroring state (left) and mirror focusing (right) over project age

Table 3: Results of the panel vector autoregression model testing proposition 1

	(1) Mirroring state$_t$		(2) Mirror focusing$_t$	
Mirroring state$_{t-1}$	.096***	(6.080)		
Mirror focusing$_{t-1}$			-.414***	(-35.272)
Project age$_t$	.000	(.055)	-.013	(-.896)
Team size$_t$	.014***	(5.779)	.020***	(5.453)
Project size$_{t-1}$	.002	(.438)	.005	(.705)
External dependencies$_{t-1}$	.062**	(2.621)	-.023	(-.512)

t statistics in parentheses, * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

4.1 Testing the First Proposition: Fluctuation of Mirroring Dynamics

Figure 6 shows the values of mirroring state (left) and mirror focusing (right) over the age of a project. The values are averaged across all projects in the dataset with confidence intervals overlaid on the plots. These model-free visualizations reveal three insights. First, Figure 6 (left) shows that mirroring state increases gradually over time but remains relatively low (less than 25%), with a lifetime average of about 14% (indicated by the dashed line). This concurs with mainstream findings in the literature about the relatively weak mirroring in OSS projects (Colfer & Baldwin, 2016). The plot shows that mirroring state tends to increase over time. Thus, OSS projects appear to coevolve mirrored project and team structures. Second, Figure 6 (right) shows that mirror focusing vacillates over time. Values above 0 (the dashed line) correspond to mirror-focusing activities, while values below 0 correspond to mirror-misting activities. Third, while fluctuations are constant, projects tend to mist the mirror earlier than later in their lifespan.

Figure 6 thus provides visual evidence supporting the baseline assertion. Another way to test the proposition is to examine the correlation between mirror focusing and its lagged value. A negative correlation reflects a fluctuation pattern in which positive values tend to reverse (focusing → misting), and vice versa (misting → focusing). Figure 7 plots two autocorrelograms for mirroring state (left) and mirror focusing (right). Confidence intervals are overlaid on top of the plots. While the left plot shows a positive, significant autocorrelation at lag 1, the right plot shows a negative, significant autocorrelation at lag 1. This visual evidence also supports proposition 1.

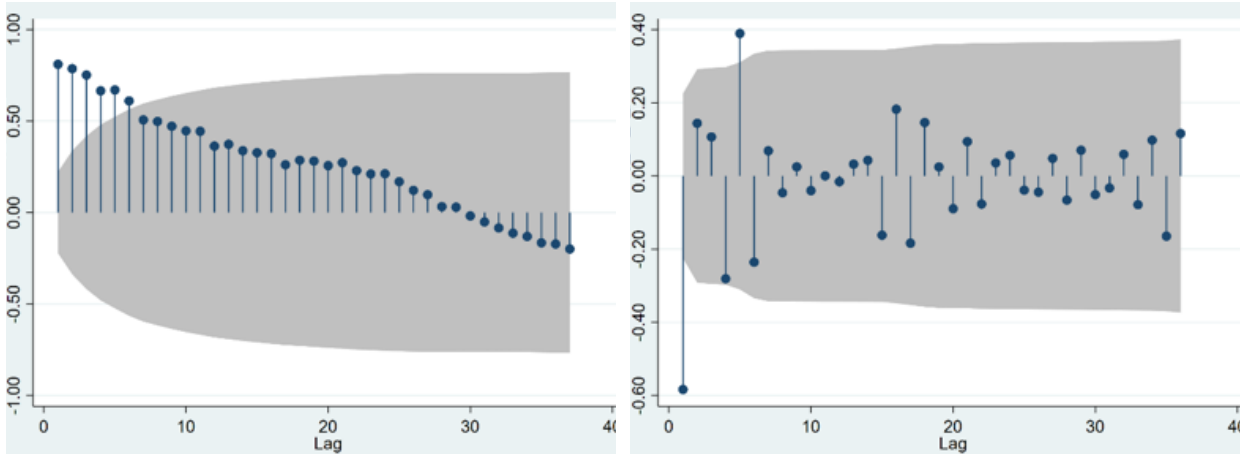


Figure 7: Auto-correlograms for mirroring state (left) and mirror focusing (right)

The two figures support proposition 1, but they combine all project panels in the data into a single time series. A statistical test of the original panel can be achieved with the panel vector autoregression model (Table 3). Specifically, the correlation between mirroring $state_{t-1}$ and mirroring $state_t$ is positive and significant (β=.096***), while the correlation between mirror $focusing_{t-1}$ and mirror $focusing_t$ is negative and significant (β=-.414***). Therefore, the first proposition is supported. When mirroring is viewed dynamically, OSS projects vacillate between misting and focusing the mirror over time.

Table 4: Results of the first stage of regression analysis testing proposition 2

	Mirroring $state_t$	
Work $specialization_{t-1}$	.027***	(8.065)
Work $centralization_{t-1}$	.080***	(8.734)
Project age_t	.008***	(8.333)
Team $size_t$	.016***	(28.163)
Project $size_t$	-.000	(-1.346)
External $dependencies_t$	.027***	(8.065)
R^2	.207	
F value	366	
Log likelihood	7939	

t statistics in parentheses, * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

4.2 Testing the Second Proposition: Mirror Focusing Promotes Project Evolution

To test proposition 2, we examine the effect of mirror $focusing_t$ on system-wide $evolution_{t+1}$ and module-level $evolution_{t+1}$ (Table 5). Stage 2 regression coefficients are positive and significant for system-wide (β=.048*) and module-level evolution (β=.098***). Interestingly, the coefficients for mirroring $state_t$ are negative and significant (β=-.179* and β=-.378***) for system-wide and

module-level evolution, respectively. These results support Proposition 2, i.e., that mirror focusing enhances evolution across both dimensions, indicating that increases in correspondence, regardless of starting level, drive subsequent evolution. These results are not incommensurable, as prior work suggests that when examining the design of systems, rather than their manufacturing as an outcome, static mirroring can be counterproductive to change (i.e., the mirroring trap). Misting the mirror can benefit complex product design (Baldwin, 2023). Our findings show that refocusing the mirror after misting it brings evolutionary benefits.

Table 5: Results of the second stage of regression analysis testing proposition 2

	(1) System-wide evolution$_{t+1}$		(2) System-wide evolution$_{t+1}$		(3) Module-level evolution$_{t+1}$		(4) Module-level evolution$_{t+1}$	
Mirroring state$_t$	-.179*	(-2.509)	-.130***	(-3.409)	-.378***	(-3.385)	-.246***	(-4.344)
Mirror focusing$_t$			.048*	(2.501)			.098***	(3.413)
Project age$_t$	.005***	(4.397)	.006***	(4.820)	-.010***	(-5.658)	-.010***	(-5.763)
Team size$_t$	.004**	(3.074)	.002***	(4.572)	.011***	(5.573)	.007***	(8.921)
Project size$_t$	-.001***	(-3.520)	-.002***	(-3.670)	-.002***	(-3.604)	-.002***	(-3.910)
External dependencies$_t$	-.154***	(-18.550)	-.156***	(-17.284)	-.161***	(-16.459)	-.166***	(-16.338)
R^2	.119		.131		.092		.122	
F value	289		206		40		302	
Log likelihood	8039		7699		291		869	

t statistics in parentheses, * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

The two dependent variables —system-wide and module-level evolution— and mirroring as a state have a range of 1 (from 0 to 1). However, mirror focusing ranges from -1 to 1 and thus has a range of 2. Thus, the linear coefficients reported in Table 5 are not only statistically significant but also practically significant. Practically, the effect sizes of mirroring state$_t$ on system-wide evolution$_{t+1}$ (β=-.130***) and module-level evolution$_{t+1}$ (β=-.246***) represent an association between the independent variable and the two dependent variables of -13% and -24%. For instance, ceteris paribus, when mirroring state$_t$ goes from its min value of 0 to its max value of 1, system-wide evolution $_{t+1}$ and module-level evolution$_{t+1}$ drop by -.13 and -.24, respectively.

The effects mirror focusing$_t$ on system-wide evolution$_{t+1}$ (β=.048*) and module-level evolution$_{t+1}$ (β=.098***) translate to an association of 9.6% and 19.6% between the independent

variable and the two dependent variables, respectively (given mirror focusing$_t$ has a range of 2). For example, ceteris paribus, when mirror focusing$_t$ goes from its min value of -1 (max misting) to its max value of 1 (max focusing), system-wide evolution $_{t+1}$ and module-level evolution$_{t+1}$ increase by .096 and -.196, respectively. Finally, due to instrumentation, all of these effects in stage 2 are likely to be underestimations of the true effects, confirming the significant and comparable effects of mirror focusing versus mirroring state on system evolution.

4.3 Testing the Third Proposition: When Does Mirror Focusing Promote Evolution?

Having established the effect of mirroring dynamics on system-wide and module-level evolution, we now delve deeper into the various pathways that mirroring dynamics can take. Table 6 and Table 7 present the split-sample results for system-wide evolution and module-level evolution, respectively. Figure 8 overlays these results on the research model. First, mirroring as a state remains negative or turns statistically nonsignificant across all eight configurations. Mirroring dynamics (focusing the mirror) is significant only through modularization (B→D and C→D) in the case of system-wide evolution. In the case of module-level evolution, it is significant through modularization (B→D and C→D) and a single monolithization configuration (B→A), which reduces the modularity of the team structure to match that of the project architecture.

Table 6: Split sample results for system-wide evolution

	(1) B→D	(2) C→D	(3) B→A	(4) C→A	(5) A→B	(6) A→C	(7) D→B	(8) D→C
Mirroring state$_t$	-.328***	-.268***	-.110	-.015	.075	-.504***	.063	-.348*
	(-4.724)	(-3.540)	(-.686)	(-.121)	(.863)	(-3.486)	(.841)	(-2.116)
Mirror focusing$_t$	.267***	.127*	.049	-.010	-.095	.009	-.083	.062
	(4.162)	(2.324)	(.894)	(-.558)	(-1.620)	(.441)	(-1.202)	(1.150)
Project age$_t$	.003	.005	.015***	.006**	.004	.005*	.002	.008
	(1.437)	(1.354)	(3.709)	(2.597)	(1.013)	(2.026)	(.880)	(1.757)
Team size$_t$	.006***	.002**	.002	.001	-.001	.009***	-.000	.005*
	(5.712)	(3.033)	(.883)	(.483)	(-.747)	(3.729)	(-.074)	(2.491)
Project size$_t$	-.001*	-.003**	-.003**	-.002*	-.001	.000	-.001	-.003
	(-2.346)	(-2.911)	(-2.733)	(-2.431)	(-.998)	(.307)	(-1.091)	(-1.706)
External dependencies$_t$	-.137***	-.063*	-.197***	-.171***	-.143***	-.153***	-.151***	-.099**
	(-9.299)	(-2.243)	(-7.337)	(-10.585)	(-4.826)	(-9.910)	(-8.899)	(-3.044)
R^2	.100	.053	.188	.174	.099	.058	.136	-.004
F value	71.729	17.560	24.735	71.756	17.432	55.778	67.979	11.514
Log likelihood	2796	1187	1119	3171	1299	2581	3296	970

t statistics in parentheses, * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

Table 7: Split sample results for module-level evolution

	(1) B→D	(2) C→D	(3) B→A	(4) C→A	(5) A→B	(6) A→C	(7) D→B	(8) D→C
Mirroring state$_t$	-.544***	-.507***	-.546*	-.164	-.117	-.552*	.069	-.146
	(-5.313)	(-4.517)	(-2.136)	(-.890)	(-.907)	(-2.558)	(.604)	(-.588)
Mirror focusing$_t$	.452***	.239**	.211*	.006	-.013	.004	-.104	-.011
	(4.739)	(2.932)	(2.454)	(.245)	(-.141)	(.144)	(-.981)	(-.137)
Project age$_t$	-.013***	-.010*	.005	-.011***	-.005	-.012***	-.014***	-.008
	(-4.395)	(-1.961)	(.824)	(-4.002)	(-.915)	(-3.452)	(-4.898)	(-1.249)
Team size$_t$	.012***	.005***	.010**	.009**	.002	.013***	.003*	.006*
	(7.319)	(3.989)	(3.120)	(2.859)	(1.556)	(3.785)	(2.206)	(2.077)
Project size$_t$	-.002*	-.002	-.004**	-.002**	-.003*	.000	-.002*	-.003*
	(-2.157)	(-1.537)	(-2.689)	(-2.668)	(-2.044)	(.181)	(-2.061)	(-2.388)
External dependencies$_t$	-.153***	-.089**	-.183***	-.189***	-.131***	-.163***	-.177***	-.088*
	(-8.654)	(-2.621)	(-6.115)	(-11.192)	(-3.940)	(-8.333)	(-9.817)	(-2.197)
R^2	.118	.083	.059	.158	.097	.096	.145	.071
F value	112.578	34.018	31.169	113.646	21.999	75.589	102.962	14.881
Log likelihood	488.228	328.382	284.356	695.557	389.353	457.214	720.185	307.539

t statistics in parentheses, * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

Second, our finding that mirror focusing is beneficial through modularization (B→D and C→D) provides the first empirical evidence of mirroring's desirable direction. The extolled virtues of mirroring as a design rule justify mirror misting only under certain circumstances (Baldwin & Clark, 2000). Our results complement this finding, namely that mirroring misting is only one phase in the process of refocusing the mirror. Finally, the only exception (B→A) suggests that a monolithic mirror is advantageous only when evolving system components, but not for accruing system-wide evolutionary benefits.

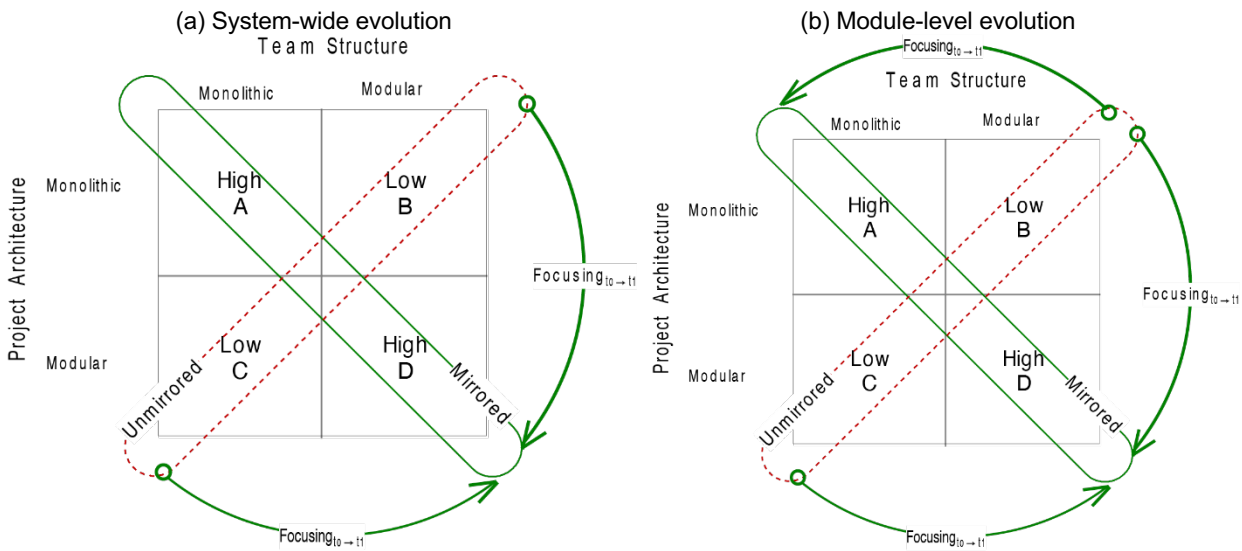


Figure 8: Visualization of split-sample analysis; green → are significant positive effects on evolution

4.4 Robustness Checks

We sampled projects with at least two distinct developers during the 2011-2016 observation window and at least one monthly commit (see footnote 4). Because we reconstruct project status quarterly, all variables are measured on a project-quarter basis (i.e., panel data setting). Our data contains 1,279 unique projects and 25,947 quarterly observations.

Team size is the number of developers who commit code to a project in a quarter. There are some projects where a single developer made commits in a particular quarter, resulting in a team size of 1 in that quarter. Because assessing mirroring is more meaningful when having multiple developers, we exclude these observations (Team Size of 1) from the data. This results in a reduced sample of 1,269 projects and 18,640 observations.

We then re-analyze the resulting smaller sample and test the three propositions for robustness. The full results are reported in the appendix. All results are robust to this alternative sample and are robust, further demonstrating the applicability of the theory to projects that tend to be smaller.

5 DISCUSSION

Mirroring refers to the correspondence between the project architecture and the team structure in terms of technical dependencies and organizational ties, respectively. We fundamentally reconceptualize mirroring from a static structural property to a dynamic phenomenon, resolving paradoxical observations about why some OSS projects thrive despite violating the mirroring hypothesis. Our analysis shows that mirroring *dynamics*—not mirroring states—drive successful evolution, introducing a temporal dimension that advances our understanding of socio-technical OSS development.

Weak mirroring in OSS has long been observed (Colfer & Baldwin, 2016). The prevalent explanation in the literature is that mirroring is unnecessary in OSS projects because they often rely on alternative, emergent coordination mechanisms that substitute for mirroring. Traditional formal organizational structures are not needed in OSS projects, as they can accomplish integration through other means, such as visibility of actions and alignment of goals (Colfer & Baldwin, 2016; Puranam et al., 2014). However, this explanation does not necessarily preclude mirroring, given that a modular codebase—as in OSS—can still be matched with a modular team.

Our contribution to this discourse is to examine the previously unexplored dynamics of mirroring in OSS projects. We developed three propositions. (1) mirroring dynamics fluctuate in OSS projects, changing between mirror-misting and mirror-focusing, (2) these dynamics promote evolution beyond the static mirroring effects, and (3) evolution is promoted when the mirror is focused to the modular-project—modular-team-state. Our results supported the three propositions. The fluid nature of OSS projects creates opportunities for developers to mist the mirror and, in so doing, subsequent opportunities to refocus it, and as a result promote the project

evolution. Our findings make several unique and novel contributions to IS research and the literature on the mirroring hypothesis.

5.1 Contributions to IS Research

a. OSS Development. Our study is the first to systematically examine the *dynamics* of mirroring in OSS. It emphasizes the interplay between team structure and project architecture in driving project evolution. Our findings suggest that the correspondence between the architecture and team structure in OSS projects is not fixed but fluctuates over time. Our study shows that this fluidity is generative, as it leads developers to gain proficiency and specialization in their tasks (G. K. Lee & Cole, 2003; Singh et al., 2011). Today, OSS has been adopted as a convenient business model in which software firms attract external contributors to their projects (von Hippel & von Krogh, 2003; Young & Fitzgerald, 2006). This is unsurprising, given that the free revealing of innovation creates significant value across industries (Hoffmann, Nagle, & Zhou, 2024; Nagle, 2019). While companies may be enticed to manage OSS projects (Ho & Rai, 2017), such direct management can deprive these projects of the benefits of experimentation and exploration. Our findings underscore the importance of bottom-up emergent dynamics in driving OSS evolution. In a fundamental departure from prior mirroring research that sought optimal mirroring configurations, this study shows that the dynamics of misting and refocusing the mirror generate evolutionary advantages. This finding introduces temporal heterogeneity as a core mechanism in socio-technical systems, suggesting that mirroring benefits arise not from achieved correspondence but potentially from the knowledge-generating process of repeated misalignment and realignment.

b. IS Evolution. This study sheds light on the poorly understood evolutionary consequences in IS development (Agarwal & Tiwana, 2015), thereby broadening the understanding of the multidimensional processes that shape information systems. IT project managers can gain

strategic benefits by recognizing the nontrivial contributions of mirroring toward achieving long-term evolutionary results.

First, our results underscore how mirroring dynamics improve evolution. Specifically, the upward phase of the vacillation cycle yields evolutionary benefits. While the downward phase of the cycle has no adverse effects that cancel out the positive effects of the upward phase, this downward phase is necessary for an upward phase to occur. Recent work has connected IS evolution with synchronizing team and project structures (Tiwana & Safadi, forthcoming). However, this work underappreciates the temporality of the process. Evolution not only requires synchronizing the two but rather resynchronization following drift. Static mirroring is detrimental to change, confirming prior findings on the mirroring trap (Baldwin, 2023). As a venue for future work, the dynamics introduced here, therefore, corroborate what Taleb (2012) characterizes as *antifragility* in systems. Unlike fragile systems that break from disturbances and robust ones that merely recover from stressors, mirror misting acts as a stressor that instead causes the system to improve. As the system self-corrects, small disturbances can lead to cumulative improvements over time. The identification of mirroring-induced antifragility represents a novel thread in the IS evolution discourse.

Second, although information systems are brittle, tend to atrophy (Tiwana & Safadi, 2024), and benefit from fewer internal interdependencies (Tiwana & Safadi, 2025), they experience ups and downs throughout their lifecycle. We identify mirroring dynamics—specifically mirror-misting and mirror-focusing—as two mechanisms undergirding IS evolution. Second, derived directly from Henderson and Clark (1990) dimensions of innovation: changes within modules and changes in how modules connect, we can spot whether mirroring dynamics differentially affect module refinement or module connections. This nuanced pattern enabled us to recognize

that mirroring advantages are reaped in a modular-project—modular-team state (confirming proposition 3), with one exception. Refocusing the mirror by restructuring a modular team to match a monolithic project is advantageous for module evolution. This finding supports prior work on IS systems, underscoring the advantages of architectural monolithicity in specific circumstances (Tiwana, 2018) and prior work on knowledge management supporting shuffling teams to match problem-solving requirements (von Hippel, 1994).

5.2 Contributions to Research on Mirroring

Our key contribution to the literature on mirroring is introducing the two interdependent aspects of mirroring—state and dynamics—by innovatively leveraging a fluid OSS project context in which these aspects are particularly visible. This dualistic conceptualization goes to the heart of the persistent tensions in mirroring research, particularly regarding the apparent exceptions to the hypothesis documented in Colfer and Baldwin (2016) and the mechanisms by which mirroring affects outcomes. Prior work examined the optimal degree of mirroring (Burton & Galvin, 2022; Hao et al., 2017; Querbes & Frenken, 2018). Burton and Galvin (2022) specifically urged attention to systematic exceptions, i.e., contexts where mirroring is absent, yet systems succeed, or present yet systems fail. Our dynamic model tackles this puzzle by demarcating temporal from cross-sectional mechanisms. Perhaps what appears to be weak static mirroring in OSS is high dynamic mirroring operating through iterative adjustment cycles rather than sustained static mirroring. The observed exceptions to mirroring might thus be a different manifestation of coordination benefits arising from the process of aligning structures, not solely from achieved mirrored states. Different degrees of mirroring may be optimal in various settings, and overly strict mirroring can be counterproductive (Colfer & Baldwin, 2016). However, the dynamics of mirroring are complex and depend on an underlying balance of costs and benefits, which may shift in the wake of technological change (Baldwin, 2023). Our findings highlight the

organizational mechanisms that enable projects to escape Baldwin's (2023) "mirroring trap." Although our study was observational, findings suggest that projects could escape this trap by employing mirror-misting to enable cross-boundary exploration and mirror-focusing to coalesce architectural learning. This temporal variation can prevent ossification that Baldwin noted, while simultaneously preserving the coordination benefits that encourage mirroring.

Prior work suggests that altering mirroring is a strategic choice (Baldwin, 2023; Burton & Galvin, 2022; Sorkun & Furlan, 2017). This study broadens the understanding of mirroring beyond a static concept that predominates previous studies and sheds light on how mirroring dynamics matter beyond static mirroring. Specifically, the effects of mirroring state and mirroring dynamics are separate and complementary, shaping evolutionary outcomes. While mirroring can trap performance (Colfer & Baldwin, 2016), these effects can be reduced by deviating and reestablishing mirroring. Heeding the call to better understand organizational dynamics (Ahuja, Soda, & Zaheer, 2012; Nickerson & Zenger, 2002), this finding underscores the need to examine both first and second-order effects of mirroring, providing a novel approach to understanding the nuanced coevolution of team and project structures.

Prior work has extensively studied the contextual factors that make mirroring desirable (e.g., Fang & Kim, 2023). The weak mirroring observed by Colfer and Baldwin (2016) was attributed to alternative OSS coordination mechanisms that reduce the need for direct communication. Our process lens, in contrast, implies that OSS projects exhibit weak mirroring when measured cross-sectionally, precisely because they continuously adjust mirroring over time. Snapshot measurements capture neither the misting activities that produce architectural knowledge nor the focusing activities that use it to drive subsequent realignment of the mirror. By theorizing mirroring as a temporal process rather than solely a static configuration, we add mirroring

dynamics as a generalizable mechanism operating in projects with dispersed authority and fluid membership. Our findings highlight the importance of considering mirroring dynamics for reconciling apparent contradictions in the mirroring literature. Static mirroring can be detrimental to performance in some settings (i.e., the mirroring trap). However, changes in mirroring (both upward and downward) can counteract such negative outcomes.

5.3 Practical Implications

The important role of tinkering and mixing has long been recognized from an organizational perspective, especially within the realm of agile and extreme programming paradigms (Ågerfalk, Fitzgerald, & Slaughter, 2009; Schwaber & Beedle, 2002). Based on our results, when the strict division of labor during development is disrupted, it can result in similar outcomes, providing an opportunity for IT project managers, especially when development is open and distributed.

5.4 Methodological Contributions

Mirroring studies have historically relied on qualitative assessments of team or organizational structure. Our novel, network-theoretic measures for project architecture and team structure are based on actual activity and use the same network measures. This approach can be leveraged in future mirroring studies to achieve both unprecedented commensurability and temporal granularity using trace data. Furthermore, unlike the vast majority of prior IS studies that measure modularity using psychometric scales, our approach represents an epistemological departure, grounding inference in the artifact itself rather than in informants' perceptions of it. We directly analyze every line of source code in every project to measure both the technical and organizational structures of the project and temporal changes in both. Very few prior studies have used source code data to measure the architectural modularity (e.g., MacCormack et al., 2006 used a sample of six projects).

5.5 Limitations and Boundary Conditions

Three sets of conditions theoretically bound the scope of our findings and limit generalizability.

a. Emergent versus intentional dynamics. Our theorizing assumes mirroring dynamics as an emergent result of coordination rather than intentional management. Every developer is focused on their direct work—such as bug fixes and features —without deliberate awareness of the overall effect on mirroring. Thus, the phenomenological experience of development is entirely local, wherein no individual actor perceives or necessarily intends the aggregate mirroring pattern that emerges from their collective actions. The evolutionary benefits we observed are an emergent result of the cumulative effects of many decentralized decisions rather than intentional mirroring. This scope distinguishes OSS settings, where dynamics emerge from self-selecting and diffuse authority, from traditional organizations, where managers can deliberately manipulate mirroring. On the other hand, our findings could generalize organizational settings with decentralized decision-making and shifting membership, such as decentralized autonomous organizations and related emerging organizational forms.

b. The potential for survival bias in the sample. We focused on successful OSS projects. Our Δ-mirroring measure captures change magnitude but not oscillation frequency or rhythm. Our quarterly observations were a pragmatic balance between granularity and computational feasibility. The negative autocorrelation in Table 3 provides indirect evidence that magnitude changes reflect oscillation dynamics. Nevertheless, higher-frequency data can more directly test frequency-based hypotheses about mirroring rhythms in future work. Since we examined only projects that survived the entire observation period, we may be overlooking those that did not make it past certain stages. This means that our findings may not represent all OSS projects, especially those facing development challenges.

c. Contingency factors. Five project attributes can plausibly affect the magnitude and direction of mirroring oscillation. First, project maturity represents a temporal boundary condition: early-stage projects are likely to exhibit unstable oscillations as architectural conventions stabilize and developer teams coalesce, whereas mature projects with known design patterns and module ownership are more likely to resist such dynamics as restructuring project-wide code becomes costlier. Second, small projects with small numbers of developers have little opportunity for extensive structural mismatch (see the robustness check and the appendix), constraining the range over which mirroring can vary. In contrast, large projects with hundreds of contributors and complex interdependencies incur high coordination costs that may render repeated codebase or team reorganization economically infeasible. Third, domain properties influence the comparative value of structural flexibility. Projects in rapidly evolving domains, such as machine learning libraries, may derive disproportionate evolutionary benefit from the exploratory potential of mirror-misting. For some in stable domains, the coordination advantages of long-term mirroring may outweigh potential gains. Fourth, even in decentralized OSS environments, projects with strong architectural control by a well-established developer (e.g., Linux) may exhibit qualitatively different oscillation patterns than peer-governance projects. The power of the former can accelerate refocusing attempts or restrict mirror misting experimentation (Shaikh & Vaast, 2016). Fifth, developer turnover can affect the balance between mirror-misting and mirror-focusing. Fluidity is generative in open collectives (Faraj, Jarvenpaa, & Majchrzak, 2011). High developer churn creates more opportunities for accidental mirror misting, since new developers lack prior knowledge of implicit project conventions and dominant patterns of labor division. Yet it constrains project developers' collective capability to refocus the mirror by

developing a meta-understanding of the project (Faraj & Sproull, 2000). These are fertile areas for additional research into mirroring dynamics.

5.6 Future Work

Our key insight—that mirroring vacillation in OSS projects promotes project evolution—is closely tied to the fluid nature of OSS projects. It will be interesting to examine how these dynamics generalize to traditional IS projects in organizations. Second, we focused on project evolution as a key important outcome (Agarwal & Tiwana, 2015). As open-source software (OSS) projects age and mature, other factors such as maintainability might become more salient. Future work should investigate the impact of mirroring in OSS projects beyond just development and how it affects various aspects of software lifecycle activities—i.e., in-design versus in-use aspects of architecture (Alochet et al., 2023; Baldwin & Clark, 2000). Finally, organizational change can exhibit a vacillation pattern (Miller & Friesen, 1984; Tushman & Romanelli, 1985). The direction of mirroring—i.e., whether organization structure follows technology structure or vice versa—has been extensively studied (Colfer & Baldwin, 2016; Conway, 1968; Sanchez & Mahoney, 1996). Complementing prior work on static mirroring, future work could investigate the relationship between second-order changes in organizational and technological structures. Relatedly, an important avenue concerns configurations in which both the technical architecture and the organizational structure are highly modular, yet the correspondence between them remains low: for example, when modular teams work on modules that do not map onto the technical structure. Investigating how mirroring dynamics operate under such conditions would deepen our understanding of the distinction between absolute modularity levels and the degree of correspondence between project and team structures. The future of mirroring research lies not in optimizing configurations but in orchestrating oscillations.

5.7 Conclusion

"An oscillation is required to set a mechanism in motion." (Rovelli, 2019, p. 62). Rovelli, a physicist, might as well be talking about mirroring in light of our findings. Static correspondence between technical and organizational structures, long prescribed by the mirroring hypothesis, represents not organizational optimality but evolutionary stasis. Through our longitudinal analysis of OSS projects, we found that socio-technical systems require oscillatory dynamics—the rhythmic departure from and return to structural alignment—to generate the evolutionary momentum.

Research in organization theory has long adopted configurational approaches focusing on the relationships between structures and outcomes (Ahuja et al., 2012). However, as we know from natural sciences, second-order effects are equally important in explaining natural phenomena. Where configurational approaches sought optimal structural arrangements, our findings show that the generative mechanism lies not in any particular configuration but in the oscillatory movement between configurations. The temporal dynamics of mirror-misting and mirror-focusing transforms structural instability from a pathology to be remedied into a lever for evolution.

We see this as an opportunity to better understand how the coevolution of team structure and project architecture affects project outcomes. Fluid OSS projects provided a natural setting to help uncover how continuous oscillation in mirroring generates evolutionary benefits that transcend those provided by static mirroring.

Our findings both enhance our understanding of OSS coordination and mirroring and shed new light on the dynamics of IS projects. The mirroring hypothesis, until now, has captured only a static snapshot of an inherently dynamic phenomenon. We advocate an epistemic pivot from

structure to process, and from state to dynamics, in our pursuit of orchestrating productive oscillations.

REFERENCES

Abrigo, M. R. M., & Love, I. (2016). Estimation of panel vector autoregression in Stata. *Stata Journal*, *16*(3), 778–804. https://doi.org/10.1177/1536867x1601600314

Acheampong, A. O. (2018). Economic growth, CO2 emissions and energy consumption: What causes what and where? *Energy Economics*, *74*, 677–692. https://doi.org/10.1016/j.eneco.2018.07.022

Agarwal, R., & Tiwana, A. (2015). Editorial — Evolvable Systems : Through the Looking Glass Evolvable Systems : Through the Looking Glass of IS. *Information Systems Research*, *26*(3), 473–479. https://doi.org/http://dx.doi.org/10.1287/isre.2015.0595

Ågerfalk, P. J., Fitzgerald, B., & Slaughter, S. A. (2009). Flexible and distributed information systems development: State of the art and research challenges. *Information Systems Research*, *20*(3), 317–328. https://doi.org/10.1287/isre.1090.0244

Ahuja, G., Soda, G., & Zaheer, A. (2012). The Genesis and Dynamics of Organizational Networks. *Organization Science*, *23*(2), 434–448.

Allison, P. D. (2009). *Fixed effects regression models* (Vol. 160). SAGE publications.

Alochet, M., MacDuffie, J. P., & Midler, C. (2023). Mirroring in production? Early evidence from the scale-up of Battery Electric Vehicles (BEVs). *Industrial and Corporate Change*, *32*(1), 61–111. https://doi.org/10.1093/icc/dtac028

Andersson, M., Lindgren, R., & Henfridsson, O. (2008). Architectural knowledge in inter-organizational IT innovation. *Journal of Strategic Information Systems*, *17*(1), 19–38. https://doi.org/10.1016/j.jsis.2008.01.002

Andriani, P., & Cattani, G. (2024). Toward an Evolutionary Theory of Shadow Options: Emergent Functions of Corning's Glass-Based Innovations. *Organization Science*, *7039*, 1–23. https://doi.org/10.1287/orsc.2023.17512

Angrist, J. D., & Pischke, J.-S. (2008). *Mostly harmless econometrics*. Princeton university press.

Arellano, M. (1987). Computing robust standard errors for within-groups estimators. *Oxford Bulletin of Economics and Statistics*, *49*(4), 431–434.

Arthur, W. B. (2009). *The nature of technology: What it is and how it evolves*. Simon and Schuster.

Baldwin, C. Y. (2023). Design rules: past and future. *Industrial and Corporate Change*, *32*(1), 11–27. https://doi.org/10.1093/icc/dtac055

Baldwin, C. Y. (2024). *Design Rules, Volume 2: How Technology Shapes Organizations*. MIT Press.

Baldwin, C. Y., & Clark, K. B. (2000). *Design rules: The power of modularity* (Vol. 1). MIT press.

Baldwin, C. Y., & Clark, K. B. (2006). The Architecture of Participation: Does Code Architecture Mitigate Free Riding in the Open Source Development Model? *Management Science*, *52*(7), 1116–1127. https://doi.org/10.1287/mnsc.1060.0546

Billinger, S., Benincasa, S., Baumann, O., Kretschmer, T., & Schumacher, T. R. (2023). Learning to search collaboratively: how dyads overcome complexity and misaligned incentives in imperfect modular decompositions. *Industrial and Corporate Change*, *32*(1), 208–233. https://doi.org/10.1093/icc/dtac029

Bolici, F., Howison, J., & Crowston, K. (2016). Stigmergic coordination in FLOSS development teams: Integrating explicit and implicit mechanisms. *Cognitive Systems Research*, *38*, 14–22. https://doi.org/10.1016/j.cogsys.2015.12.003

Booch, G. (1990). *Object-oriented analysis and design*. Addison-Wesley.

Borgatti, S. P., & Everett, M. G. (2006). A Graph-theoretic perspective on centrality. *Social Networks*, *28*(4), 466–484. https://doi.org/10.1016/j.socnet.2005.11.005

Breusch, T. S., & Pagan, A. R. (1979). A simple test for heteroscedasticity and random coefficient variation. *Econometrica: Journal of the Econometric Society*, 1287–1294.

Brusoni, S., Henkel, J., Jacobides, M. G., Karim, S., MacCormack, A., Puranam, P., & Schilling, M. (2023). The power of modularity today: 20 years of "Design Rules." *Industrial and Corporate Change*, *32*(1), 1–10. https://doi.org/10.1093/icc/dtac054

Brusoni, S., Prencipe, A., & Pavitt, K. (2001). Knowledge specialization, organizational coupling, and the boundaries of the firm: why do firms know more than they make? *Administrative Science Quarterly*, *46*(4), 597–621.

Burton, N., & Galvin, P. (2022). Modularity, value and exceptions to the mirroring hypothesis. *Journal of Business Research*, *151*(March 2021), 635–650. https://doi.org/10.1016/j.jbusres.2022.07.023

Chapin, N., amd Khaled Md. Khan, J. E. H., Ramil, J. F., & Tan, W.-G. (2003). Types of software evolution and software maintenance. *Journal on Software Maintenance and Evolution: Research Practice*, *13*(July 2000), 3–30.

Chen, Y., Chowdhury, S. D., & Donada, C. (2019). Mirroring hypothesis and integrality: Evidence from Tesla Motors. *Journal of Engineering and Technology Management - JET-M*, *54*(September), 41–55. https://doi.org/10.1016/j.jengtecman.2019.09.001

Colfer, L. J., & Baldwin, C. Y. (2016). The mirroring hypothesis: theory, evidence, and exceptions. *Industrial and Corporate Change*, *25*(5), 709–738. https://doi.org/10.1093/icc/dtw027

Constantinides, P., Henfridsson, O., & Parker, G. G. (2018). Platforms and infrastructures in the digital age. *Information Systems Research*, *29*(2), 381–400. https://doi.org/10.1287/isre.2018.0794

Conway, M. E. (1968). How do committees invent. *Datamation*, *14*(4), 28–31.

Drukker, D. M. (2003). Testing for serial correlation in linear panel-data models. *The Stata Journal*, *3*(2), 168–177.

Eaton, B., Elaluf-Calderwood, S., Sørensen, C., & Yoo, Y. (2015). Distributed Tuning of Boundary Resources: The Case of Apple's iOS Service System. *MIS Quarterly*, *39*(1), 217–243. https://doi.org/10.25300/MISQ/2015/39.1.10

Fang, C., & Kim, J. J. (2023). Is modularity robust to misfits? A formal test. *Industrial and Corporate Change*, *32*(1), 47–60. https://doi.org/10.1093/icc/dtac031

Faraj, S., Jarvenpaa, S. L., & Majchrzak, A. (2011). Knowledge Collaboration in Online Communities. *Organization Science*, *22*(5), 1224–1239. https://doi.org/10.1287/orsc.1100.0614

Faraj, S., & Sproull, L. (2000). Coordinating expertise in software development teams. *Management Science*, *46*(12), 1554–1568.

Foster, L., Grim, C., Haltiwanger, J. C., & Wolf, Z. (2021). Innovation, Productivity Dispersion, and Productivity Growth. In *Measuring and Accounting for Innovation in the Twenty-First Century* (p. 103). University of Chicago Press.

Freeman, L. C. (1979). Centrality in social networks conceptual clarification. *Social Networks*,

1(3), 215–239.
Furlan, A., Cabigiosu, A., & Camuffo, A. (2014). When the mirror gets misted up: Modularity and technological change. *Strategic Management Journal*, *35*(6), 789–807. https://doi.org/10.1002/smj.2138
Fürstenau, D., Baiyere, A., & Kliewer, N. (2019). A dynamic model of embeddedness in digital infrastructures. *Information Systems Research*, *30*(4), 1319–1342. https://doi.org/10.1287/isre.2019.0864
Fürstenau, D., Baiyere, A., Schewina, K., Schulte-althoff, M., Rothe, H., Daniel, F., … Schulte-althoff, M. (2023). Extended Generativity Theory on Digital Platforms. *Information System Research*, (March). https://doi.org/10.1287/isre.2023.1209
Gousios, G., & Spinellis, D. (2012). GHTorrent: Github's data from a firehose. In *IEEE International Working Conference on Mining Software Repositories* (pp. 12–21). https://doi.org/10.1109/MSR.2012.6224294
Hao, B., Feng, Y., & Frigant, V. (2017). Rethinking the 'mirroring' hypothesis: implications for technological modularity, tacit coordination, and radical innovation. *R and D Management*, *47*(1), 3–16. https://doi.org/10.1111/radm.12106
Henderson, R. M., & Clark, K. B. (1990). Architectural innovation: The reconfiguration of existing product technologies and the failure of established firms. *Administrative Science Quarterly*, *35*(1), 9–30. https://doi.org/Article
Ho, S. Y., & Rai, A. (2017). Continued Voluntary Participation Intention in Firm-Participating Open Source Software Projects. *Information Systems Research*, (July), isre.2016.0687. https://doi.org/10.1287/isre.2016.0687
Hoetker, G. (2006). Do modular products lead to modular organizations? *Strategic Management Journal*, *27*(6), 501–518. https://doi.org/10.1002/smj.528
Hoffmann, M., Nagle, F., & Zhou, Y. (2024). The Value of Open Source Software. *SSRN Electronic Journal*. https://doi.org/10.2139/ssrn.4693148
Howison, J., & Crowston, K. (2014). Collaboration through open superposition: A theory of the open source way. *Mis Quarterly*, *38*(1), 29–50.
Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., & Damian, D. (2016). An in-depth study of the promises and perils of mining GitHub. *Empirical Software Engineering*, *21*(5), 2035–2071. https://doi.org/10.1007/s10664-015-9393-5
Keane, M. P., & Runkle, D. E. (1992). On the Estimation of Panel-Data Models Correlation When Strictly Exogenous. *Journal of Business & Economic Statistics*, *10*(1), 1–9. https://doi.org/10.1104/pp.010332.1
Kemerer, C. F. C. F., & Slaughter, S. (1999). An empirical approach to studying software evolution. *IEEE Transactions on Software Engineering*, *25*(4), 493–509. https://doi.org/10.1109/32.799945
Kennedy, P. (1992). *A Guide to Econometrics*. *MIT Press*.
Kennedy, P. (2008). *A guide to econometrics* (6th ed.). John Wiley & Sons.
Kleibergen, F., & Paap, R. (2006). Generalized reduced rank tests using the singular value decomposition. *Journal of Econometrics*, *133*(1), 97–126.
Lawrence, P. R., & Lorsch, J. W. (1967). Differentiation and Integration in Complex Organizations. *Administrative Science Quarterly*, *12*(1), 1. https://doi.org/10.2307/2391211
Lee, G. K., & Cole, R. E. (2003). From a Firm-Based to a Community-Based Model of Knowledge Creation: The Case of the Linux Kernel Development. *Organization Science*, *14*(6), 633–649. https://doi.org/10.1287/orsc.14.6.633.24866

Lee, J., & Berente, N. (2012). Digital Innovation and the Division of Innovative Labor: Digital Controls in the Automotive Industry. *Organization Science*, *23*(5), 1428–1447. https://doi.org/10.1287/orsc.1110.0707

Lehman, M. M. (1979). On understanding laws, evolution, and conservation in the large-program life cycle. *The Journal of Systems and Software*, *1*(C), 213–221. https://doi.org/10.1016/0164-1212(79)90022-0

Lehman, M. M., Ramil, J. F., Wernick, P. D., Perry, D. E., & Turski, W. M. (1997). Metrics and laws of software evolution - the nineties view. *International Software Metrics Symposium, Proceedings*, 20–32. https://doi.org/10.1109/metric.1997.637156

Lerner, J., & Tirole, J. (2002). Some simple economics of open source. *The Journal of Industrial Economics*, *L*(2), 197–234. https://doi.org/http://www.blackwellpublishing.com/journal.asp?ref=0022-1821

Lindberg, A., Berente, N., Gaskin, J., & Lyytinen, K. (2016). Coordinating Interdependencies in Online Communities: A Study of an Open Source Software Project. *Information Systems Research*, isre.2016.0673. https://doi.org/10.1287/isre.2016.0673

MacCormack, A., Baldwin, C., & Rusnak, J. (2012). Exploring the duality between product and organizational architectures: A test of the "mirroring" hypothesis. *Research Policy*, *41*(8), 1309–1324. https://doi.org/10.1016/j.respol.2012.04.011

MacCormack, A., Rusnak, J., & Baldwin, C. Y. (2006). Exploring the structure of complex software designs: An empirical study of open source and proprietary code. *Management Science*, *52*(7), 1015–1030.

Medappa, P. K., & Srivastava, S. C. (2019). Does superposition influence the success of floss projects? An examination of open-source software development by organizations and individuals. *Information Systems Research*, *30*(3), 764–786. https://doi.org/10.1287/isre.2018.0829

Miller, D., & Friesen, P. H. (1984). A Longitudinal Study of the Corporate Life Cycle. *Management Science*, *30*(10), 1161–1183. https://doi.org/10.1287/mnsc.30.10.1161

Monge, P. R., & Contractor, N. S. (2001). Emergence of communication networks. *The New Handbook of Organizational Communication: Advances in Theory, Research, and Methods*, 440–502.

Murase, T., Carter, D. R., DeChurch, L. A., & Marks, M. A. (2014). Mind the gap: The role of leadership in multiteam system collective cognition. *Leadership Quarterly*, *25*(5), 972–986. https://doi.org/10.1016/j.leaqua.2014.06.003

Murmann, J. P., & Schuler, B. A. (2023). Exploring the structure of internal combustion engine and battery electric vehicles: implications for the architecture of the automotive industry. *Industrial and Corporate Change*, *32*(1), 129–154. https://doi.org/10.1093/icc/dtac049

Nagle, F. (2019). Open source software and firm productivity. *Management Science*, *65*(3), 1191–1215. https://doi.org/10.1287/mnsc.2017.2977

Newman, M. E. J. (2006). Modularity and community structure in networks. *Proceedings of the National Academy of Sciences*, *103*(23), 8577–8582. https://doi.org/10.1073/pnas.0601602103

Newman, M. E. J., & Girvan, M. (2004). Finding and evaluating community structure in networks. *Physical Review E*, *69*(2), 026113. https://doi.org/10.1103/PhysRevE.69.026113

Nickerson, J. A., & Zenger, T. R. (2002). Being Efficiently Fickle: A Dynamic Theory of Organizational Choice. *Organization Science*, *13*(5), 547–566. https://doi.org/10.1287/orsc.13.5.547.7815

Parker, G. G., & Anderson, E. G. (2002). From buyer to integrator: The transformation of the supply-chain manager in the vertically disintegrating firm. *Production and Operations Management*, *11*(1), 75–91. https://doi.org/10.1111/j.1937-5956.2002.tb00185.x

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, *15*(12), 1053–1058.

Puranam, P., Alexy, O., & Reitzig, M. (2014). What's "New" About New Forms of Organizing? *Academy of Management Review*, *39*(2), 162–180. https://doi.org/10.5465/amr.2011.0436

Puranam, P., Raveendran, M., & Knudsen, T. (2012). Organization Design: The Epistemic Interdependence Perspective. *Academy of Management Review*, *37*(3), 419–440. https://doi.org/10.5465/amr.2010.0535

Querbes, A., & Frenken, K. (2018). Grounding the "mirroring hypothesis": Towards a general theory of organization design in New Product Development. *Journal of Engineering and Technology Management*, *47*(December 2017), 81–95. https://doi.org/10.1016/j.jengtecman.2018.01.001

Raveendran, M., Puranam, P., & Warglien, M. (2022). Division of Labor Through Self-Selection. *Organization Science*, *33*(2), 810–830. https://doi.org/10.1287/orsc.2021.1449

Roberts, J. a., Hann, I.-H. H., & Slaughter, S. a. (2006). Understanding the motivations, participation, and performance of open source software developers: A longitudinal study of the Apache projects. *Management Science*, *52*(7), 984–999. https://doi.org/10.1287/mnsc.1060.0554

Rolland, K. H., Mathiassen, L., & Rai, A. (2018). Managing digital platforms in user organizations: The interactions between digital options and digital debt. *Information Systems Research*, *29*(2), 419–443. https://doi.org/10.1287/isre.2018.0788

Roodman, D. (2009). How to do xtabond2: An introduction to difference and system GMM in Stata. *Stata Journal*, *9*(1), 86–136.

Rovelli, C. (2019). *The order of time*. Penguin.

Sabherwal, R., Sabherwal, S., Havaknor, T., & Steelman, Z. (2019). How Does Strategic Alignment Affect Firm Performance? The Roles of Information Technology Investment and Environmental Uncertainty. *MIS Quarterly*, *43*(2), 453–474. https://doi.org/10.25300/MISQ/2019/13626

Sanchez, R., Galvin, P., & Bach, N. (2023). How design rules emerge and evolve: a coevolutionary architectural perspective on firm and industry organization. *Industrial and Corporate Change*, *32*(1), 28–46. https://doi.org/10.1093/icc/dtac052

Sanchez, R., & Mahoney, J. T. (1996). Modularity, flexibility, and knowledge management in product and organization design. *Strategic Management Journal*, *17*(S2), 63–76.

Schaffer, M. E. (2015). xtivreg2: Stata module to perform extended IV/2SLS, GMM and AC/HAC, LIML and k-class regression for panel data models. *Statistical Software Components*.

Schilling, M. A. (2000). Toward a General Modular Systems Theory and Its Application to Interfirm Product Modularity. *Academy of Management Review*, *25*(2), 312–334. https://doi.org/10.5465/amr.2000.3312918

Schumpeter, J. A. (1942). *Capitalism, socialism and democracy*. New York, London: Harper & Brothers.

Schwaber, K., & Beedle, M. (2002). *Agile software development with Scrum* (Vol. 1). Prentice Hall Upper Saddle River.

Shaikh, M., & Henfridsson, O. (2017). Governing open source software through coordination

processes. *Information and Organization*, *27*(2), 116–135. https://doi.org/10.1016/j.infoandorg.2017.04.001
Shaikh, M., & Vaast, E. (2016). Folding and Unfolding: Balancing Openness and Transparency in Open Source Communities. *Information Systems Research*, *27*(4), 813–833. https://doi.org/10.1287/isre.2016.0646
Simon, H. A. (2002). Near decomposability and the speed of evolution. *Industrial and Corporate Change*, *11*(3), 587–599. https://doi.org/10.1093/icc/11.3.587
Singh, P. V., Tan, Y., & Youn, N. (2011). A hidden Markov model of developer learning dynamics in open source software projects. *Information Systems Research*, *22*(4), 790–807. https://doi.org/http://dx.doi.org/10.1287/isre.1100.0308
Sorkun, M. F., & Furlan, A. (2017). Product and Organizational Modularity: A Contingent View of the Mirroring Hypothesis. *European Management Review*, *14*(2), 205–224. https://doi.org/10.1111/emre.12101
Sosa, M. E., Eppinger, S. D., & Rowles, C. M. (2004). The Misalignment of Product Architecture and Organizational Structure in Complex Product Development. *Management Science*, *50*(12), 1674–1689. https://doi.org/10.1287/mnsc.1040.0289
Srikanth, K., & Puranam, P. (2011). Integrating distributed work: comparing task design, communication, and tacit coordination mechanisms. *Strategic Management Journal*, *32*(8), 849–875. https://doi.org/10.1002/smj.908
Stock, J. H., Wright, J. H., & Yogo, M. (2012). A survey of weak instruments and weak identification in generalized method of moments. *Journal of Business & Economic Statistics*.
Stock, J. H., & Yogo, M. (2002). *Testing for weak instruments in linear IV regression*. National Bureau of Economic Research Cambridge, Mass., USA.
Taleb, N. N. (2012). *Antifragile: Things that gain from disorder*. Random House.
Thompson, J. D. (1967). *Organizations in action: Social science bases of administrative theory*. McGraw-Hill.
TIOBE. (2020). TIOBE Programming Community Index. Retrieved October 10, 2020, from https://www.tiobe.com/tiobe-index/
Tiwana, A. (2015). Platform desertion by app developers. *Journal of Management Information Systems*, *32*(4), 40–77. https://doi.org/10.1080/07421222.2015.1138365
Tiwana, A. (2018). Platform Synergy: Architectural Origins and Competitive Consequences. *Information Systems Research*, *29*(4), 829–848. https://doi.org/10.1287/isre.2017.0739
Tiwana, A. (2025). Exaptation in Platforms: A Theory of Origins, Mechanisms, and Consequences. *Information Systems Research*, (September). https://doi.org/10.1287/isre.2022.0484
Tiwana, A., Konsynski, B., & Bush, A. a. (2010). Research Commentary —Platform Evolution: Coevolution of Platform Architecture, Governance, and Environmental Dynamics. *Information Systems Research*, *21*(4), 675–687. https://doi.org/10.1287/isre.1100.0323
Tiwana, A., & Safadi, H. (2024). Atrophy in Aging Systems: Evidence, Dynamics, and Antidote. *Information Systems Research*, *35*(1), 66–86. https://doi.org/10.1287/isre.2023.1218
Tiwana, A., & Safadi, H. (2025). Silence Inside Systems: Roots and Generativity Consequences. *Information Systems Research*, (June). https://doi.org/10.1287/isre.2022.0586
Tiwana, A., & Safadi, H. (2026). Synchronizing Development Teams with Drifting Software Systems: A Decomposability-Based Theory of Evolutionary Mechanisms. *Journal of Management Information Systems*, *43*(1), 67–108.

https://doi.org/10.1080/07421222.2025.2602394
Tushman, M. L., & Romanelli, E. (1985). Organizational evolution: A metamorphosis model of convergence and reorientation. *Research in Organizational Behavior*.
Ulrich, K. (1995). The role of product architecture in the manufacturing firm. *Research Policy*, *24*(3), 419–440.
von Hippel, E. (1994). "Sticky Information" and the Locus of Problem Solving: Implications for Innovation. *Management Science*, *40*(4), 429–439. https://doi.org/10.1287/mnsc.40.4.429
von Hippel, E., & von Krogh, G. (2003). Open source software and the "private-collective" innovation model: Issues for organization science. *Organization Science*, *14*(2), 209–223.
Weick, K. E., & Roberts, K. H. (1993). Collective mind in organizations: Heedful interrelating on flight decks. *Administrative Science Quarterly*, 357–381.
Windrum, P., Haynes, M., & Thompson, P. (2019). "breaking the mirror": Interface innovation and market capture by Japanese professional camera firms, 1955-1974. *Industrial and Corporate Change*, *28*(5), 1029–1056. https://doi.org/10.1093/icc/dtz003
Young, L., & Fitzgerald, B. (2006). *The power of language: How discourse influences society*. Equinox Pub.
Zaheer, S., Albert, S., & Zaheer, A. (1999). Time Scales and Organizational Theory. *Academy of Management Review*, *24*(4), 725–741. https://doi.org/10.5465/amr.1999.2553250

APPENDIX: ROBUSTNESS CHECKS

Below, we include the results of analyses testing proposition 1 (Table 8), proposition 2 (Table 9), and proposition 3 (Table 10 and Table 11), when project-quarter observations with a single developer are dropped, resulting in a reduced sample of 18,640 observations of 1,269 projects.

Table 8: Robustness check of the panel vector autoregression model testing proposition 1

	(1) Mirroring state$_t$		(2) Mirror focusing$_t$	
Mirroring state$_{t-1}$	.095***	(6.043)		
Mirror focusing$_{t-1}$			-.414***	(-35.271)
Project age$_t$	.000	(.043)	-.013	(-.897)
Team size$_t$	.015***	(5.836)	.020***	(5.452)
Project size$_{t-1}$	.002	(.451)	.005	(.707)
External dependencies$_{t-1}$	.062**	(2.620)	-.023	(-.512)

t statistics in parentheses, * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

Table 9: Robustness check of the second stage of regression analysis testing proposition 2

	(1) System-wide evolution$_{t+1}$		(2) System-wide evolution$_{t+1}$		(3) Module-level evolution$_{t+1}$		(4) Module-level evolution$_{t+1}$	
Mirroring state$_t$	-.234**	(-2.998)	-.146***	(-3.744)	-.448***	(-3.645)	-.271***	(-4.574)
Mirror focusing$_t$			.055**	(2.809)			.111***	(3.672)
Project age$_t$	.010***	(7.190)	.009***	(7.252)	-.003	(-1.580)	-.005**	(-2.893)
Team size$_t$	.004***	(3.390)	.002***	(4.509)	.010***	(5.253)	.007***	(8.013)
Project size$_t$	-.001**	(-3.156)	-.001**	(-3.051)	-.002***	(-3.792)	-.002***	(-3.625)
External dependencies$_t$	-.170***	(-18.135)	-.170***	(-18.619)	-.174***	(-15.797)	-.177***	(-17.104)
R^2	.106		.143		.058		.118	
F value	230.7		196.4		310.5		270.9	
Log likelihood	7385.5		7716.6		726.9		1185.7	

t statistics in parentheses, * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

Table 10: Robustness check of the split sample results for system-wide evolution testing proposition 3

	(1) B→D	(2) C→D	(3) B→A	(4) C→A	(5) A→B	(6) A→C	(7) D→B	(8) D→C
Mirroring state$_t$	-.324***	-.290***	-.079	.001	.068	-.422**	.013	-.359*
	(-4.700)	(-3.965)	(-.529)	(.007)	(.771)	(-3.093)	(.168)	(-2.190)
Mirror focusing$_t$	.256***	.155**	.034	-.014	-.068	.015	-.033	.055
	(4.045)	(2.923)	(.677)	(-.776)	(-1.205)	(.747)	(-.488)	(1.022)
Project age$_t$	.006**	.006	.015***	.009***	.005	.007**	.006**	.010*
	(2.638)	(1.609)	(3.661)	(4.070)	(1.369)	(2.751)	(2.959)	(2.009)
Team size$_t$	.005***	.002**	.001	.001	-.001	.007***	.001	.005*
	(5.293)	(3.211)	(.672)	(.427)	(-.762)	(3.309)	(.632)	(2.553)
Project size$_t$	-.001	-.003*	-.003**	-.002*	-.000	.000	-.001	-.003
	(-1.744)	(-2.312)	(-2.660)	(-2.458)	(-.265)	(.291)	(-.741)	(-1.851)
External dependencies$_t$	-.148***	-.065*	-.193***	-.182***	-.160***	-.169***	-.173***	-.105**
	(-9.553)	(-2.322)	(-6.667)	(-11.154)	(-5.752)	(-10.44)	(-10.81)	(-3.197)
R^2	.108	.035	.196	.183	.121	.091	.158	-.004
F value	64.0	16.1	21.3	65.2	17.1	56.3	67.3	12.8
Log likelihood	2708.2	1246.5	1162.9	3022.4	1423.2	2533.3	3259.3	973.8

t statistics in parentheses, * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

Table 11: Robustness Check of the split sample results for module-level evolution testing Proposition 3

	(1) B→D	(2) C→D	(3) B→A	(4) C→A	(5) A→B	(6) A→C	(7) D→B	(8) D→C
Mirroring state$_t$	-.553***	-.551***	-.507*	-.144	-.098	-.469*	.001	-.176
	(-5.310)	(-4.855)	(-2.059)	(-.779)	(-.724)	(-2.233)	(.009)	(-.716)
Mirror focusing$_t$	.439***	.275***	.187*	.009	-.000	.009	-.030	-.021
	(4.570)	(3.316)	(2.261)	(.317)	(-.000)	(.314)	(-.269)	(-.269)
Project age$_t$	-.010**	-.009	.006	-.007*	-.004	-.008*	-.009**	-.005
	(-2.926)	(-1.673)	(1.023)	(-2.316)	(-.805)	(-2.099)	(-3.124)	(-.802)
Team size$_t$	.011***	.005***	.009**	.008*	.002	.011**	.004*	.005*
	(6.482)	(3.746)	(2.927)	(2.566)	(1.164)	(3.247)	(2.218)	(2.069)
Project size$_t$	-.001	-.001	-.003*	-.002**	-.002	.000	-.002	-.004**
	(-1.540)	(-.986)	(-2.540)	(-2.797)	(-1.516)	(.389)	(-1.932)	(-2.615)
External dependencies$_t$	-.161***	-.086*	-.189***	-.201***	-.147***	-.182***	-.191***	-.096*
	(-8.670)	(-2.423)	(-5.961)	(-11.274)	(-4.493)	(-9.144)	(-10.19)	(-2.422)
R^2	.114	.063	.072	.159	.096	.108	.146	.074
F value	96.8	30.7	27.9	100.4	20.6	68.6	91.5	15.0
Log likelihood	579.0	334.4	308.3	760.6	416.5	599.9	806.7	329.4

t statistics in parentheses, * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$